



A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses

Majd Soud¹ · Grischa Liebel¹ · Mohammad Hamdaqa^{1,2}

Accepted: 26 September 2023 / Published online: 30 November 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Context Smart contracts are programs that are automatically executed on the blockchain. Code weaknesses in their implementation have led to severe loss of cryptocurrency. It is essential to understand the nature of code weaknesses in Ethereum smart contracts to prevent them in the future. Existing classifications are limited in several ways, e.g., in the breadth of data sources, and the generality of proposed categories.

Objective We aim to characterize code weaknesses in Ethereum smart contracts written in Solidity, and provide an overview of existing classification schemes in relation to this characterization.

Method We extracted code weaknesses in Ethereum smart contracts from two public coding platforms and two vulnerability databases and categorized them using an open card sorting approach. We devised a classification scheme of smart contract code weaknesses according to their error source and impact. Afterwards, we mapped existing classification schemes to our classification.

Results The resulting classification consists of 11 categories describing the error source of code weaknesses and 13 categories describing potential impacts. Our findings show that the language specific coding and the structural data flow categories are the dominant categories, but that the frequency of occurrence differs substantially between the data sources.

Conclusions Our findings enable researchers to better understand smart contract code weaknesses by defining various dimensions of the problem and supporting our classification with mappings with literature-based classifications and frequency distributions of the defined categories.

Communicated by: Xin Xia

✉ Majd Soud
majd18@ru.is

Grischa Liebel
grischal@ru.is

Mohammad Hamdaqa
mhamdaqa@ru.is; mhamdaqa@polymtl.ca

¹ Department of Computer Science, Reykjavik University, Reykjavik, Iceland

² Department of Computer and Software Engineering, Polytechnique Montreal, Montreal, Quebec, Canada

Keywords Blockchain · Smart contracts · Ethereum · Solidity · Software security · Code weakness

1 Introduction

To process autonomous tasks, some blockchains execute digital programs called *smart contracts* that become immutable once deployed (Zheng et al. 2020). Smart contracts are autonomous programs that can (1) customize contracting rules and functions between contractors and (2) facilitate transferring irreversible and traceable digital cryptocurrency transactions (Hewa et al. 2020). Smart contracts were born and continued to grow with noteworthy achievements in various life fields, including industry, finance, and economics. Smart contracts and blockchains involve a variety of software design and development decisions (Sillaber et al. 2021; Khan et al. 2021). For instance, decisions about the logic and development process of a contract, as well as interrelation design decisions such as the dependencies and interactions between contracts and libraries after deployment.

Due to the immaturity of blockchain technology, code weaknesses in smart contracts can have severe consequences and result in substantial financial losses. For instance, the infamous DAO attack in 2016 Mehar et al. (2019) resulted in stealing around 50 million dollars because of exploiting a re-entrancy vulnerability in the Distributed Autonomous Organizations (DAO) contract¹. Therefore, understanding code weaknesses in smart contracts is critical to perceive the threats they represent, e.g., to develop predictive models or software engineering tools that can predict or detect threats with a high precision (Seacord and Householder 2005).

One suitable way to support understanding in a domain is to structure its knowledge through taxonomies and classification schemes (Vegas et al. 2009; Ralph 2018). A classification scheme of code weaknesses in smart contracts is essential, as (i) it can raise awareness among researchers and practitioners about the different categories of code weaknesses in smart contracts, (ii) it helps in the decision-making process and development of smart contract testing and security analysis tools, detection methods, verification methods, and validation approaches, and (iii) it provides a common vocabulary that facilitates communication and focused discussions about contracts' quality in relation to specific architectures or standards, such as ERC token standards (Norvill et al. 2019).

Existing studies attempt to classify code weaknesses in Ethereum smart contracts (Atzei et al. 2017; Dingman et al. 2019; Chen et al. 2020; Zhang et al. 2020; Rameder 2021). While they provide valuable insights into existing security issues in smart contracts, they have the following limitations:

1. **Several studies mix different classification dimensions.** This can lead to classifications where categories overlap, which is problematic if one data point (e.g., a smart contract bug) is assigned to one category only. Overlapping categories introduce ambiguity that limit the utility of these taxonomies (Ralph 2018). For instance, in Zhang et al.'s classification (Zhang et al. 2020) each smart contract bug is classified into one out of nine categories (with several sub-categories each). However, bugs often fit into multiple categories. For example, bugs in the logic sub-category "Denial of Service" can also affect security and would thus fit into both categories. The classification does not clarify how to proceed in this and similar cases. This effectively limits the usability of the classification.
2. **Code weaknesses are classified into broad categories.** Several studies classify code weaknesses into broad categories. However, broad categories are problematic, as the

¹ <https://www.coindesk.com/understanding-dao-hack-journalists>

membership in a category does not allow to reason about a code weakness's properties (Ralph 2018). For example, Dingman et al.'s defect and vulnerability classification (Dingman et al. 2019) contains four categories, including "security". These categories are so broad that they do not provide much information about instances contained therein.

3. **Data sources differ widely.** Several existing smart contract vulnerability classifications rely only on code weaknesses published in academic literature or white literature, e.g., Atzei et al. (2016) and Alharby and Van Moorsel (2017), while Chen et al. (2020) used posts on StackExchange, and Zhang et al. (2020) used a mix of academic literature and GitHub project data. However, biases in data sources, such as representing only part of the overall population of smart contract development, can affect how representative and complete the resulting classifications are. Therefore, existing classifications cannot be compared.
4. **Important data sources are omitted in existing classifications.** To our knowledge, no existing classification scheme or taxonomy for smart contract code weaknesses uses established vulnerability and defect registries such as the Smart Contract Weakness Registry (SWC)² and code weaknesses in the Common Vulnerability and Exposure (CVE)³. Hence, existing classifications likely suffer from sampling biases.

Therefore, further work is necessary to achieve precise classifications of smart contract code weaknesses. Using fine-grained classes can prevent critical information about smart contract code weaknesses from being missed or lost and improve consistency of analyses among software engineers and experts, especially when evaluating potential security incidents. Hence, this paper aims to characterize code weaknesses in Ethereum smart contracts and present an overview of the related existing classification schemes in relation to the proposed characterization, and devise a unified, generally applicable classification scheme.

The contribution of this study is two-fold:

1. To complement existing studies by classifying smart contract code weaknesses extracted from a variety of important data sources according to the different dimensions presented in existing work.
2. To provide an overview of existing classifications, outlining the different classification dimensions, and mapping existing classifications to our classification scheme using error source and impact as dimensions of the code weaknesses.

We extracted and manually analyzed data related to Ethereum smart contracts written in Solidity from four data sources, i.e., 2065 Stack Overflow posts, 3160 GitHub issues, and 523 and 37 weaknesses from CVE and SWC respectively. Using an open card sorting approach, we devised a classification scheme that uses the error source and impact of a code weakness as dimensions. Furthermore, we mapped existing classifications to this scheme and analyzed the frequency distribution of the defined categories per data source, to reason about potential biases in existing data sources. The resulting classification scheme consists of 11 categories describing the error source, and 13 categories describing potential impacts. We make the data and corresponding analyses publicly available at Soud et al. (2021).

Our findings show that most existing classifications use broad categories that are applicable to many code weaknesses and overly generic, such as "security" or "availability". In our classification, language specific coding and structural data flow categories are dominant code weaknesses in Ethereum smart contracts. However, frequency distributions of the error

² <https://swcregistry.io/>

³ <https://cve.mitre.org/>

source categories differ widely across data sources. This indicates that classifications based on a single data source should be treated with caution.

The remainder of this paper is organized as follows. Section 2 presents the background on Ethereum smart contracts, Solidity and the used data sources. Section 3 discusses how existing work relates to our study and what gaps exist. Section 4 describes the methodology that we followed. In Section 5, we report our findings in terms of the obtained classification and mapping to existing work. We then discuss our findings in Section 6 and evaluate the classification by argument in Section 7. Finally, potential threats to validity are reported in Section 8 and the paper is concluded in Section 9.

2 Background

In this section, we discuss the background of our work. Specifically, we discuss definitions used in this paper, the Ethereum environment, existing vulnerability and weakness databases, smart contract security in general, and card sorting techniques.

2.1 Definitions

Before we can discuss a classification scheme for smart contract code weaknesses, it is fundamental that we have a sufficiently specific definition of what we are classifying. There have been efforts to formally define concepts such as vulnerability and code weaknesses in SE. However, we will use the formal definition of a vulnerability and weakness that were defined in the Ethereum Improvement Proposals (EIPs)⁴ as follows:

- **Vulnerability:** “A weakness or multiple weaknesses which directly or indirectly lead to an undesirable state in a smart contract system” (Ethereum Improvement Proposals (2020) 1470). A vulnerable contract does not necessarily imply exploited (Perez and Livshits 2019). Moreover, as the contract is a digital agreement between two or more parties, exploiting the contract is not always done by external malicious actors. A venerable contract can also be exploited by one of the contracting parties such as the contract owner who can use vulnerabilities to gain more profits such as in CVE-2018-13783⁵. Any of the contractors can also exploit it, the miners or even the developers who implemented the contract (e.g., CVE-2018-17968⁶).
- **Weakness:** “a software error or mistake in contract code that in the right conditions can by itself or coupled with other weaknesses lead to a vulnerability” (Ethereum Improvement Proposals (2020)).

What distinguishes smart contract code weaknesses from other software applications is that any smart contract code instruction costs a specific amount of gas (see Section-2.4). This means even if the weakness was not exploited, it would result in losing Ether⁷ when it is triggered by the contract itself and executed, which makes the contract vulnerable even if it is not exploited.

⁴ <https://eips.ethereum.org/>

⁵ <https://nvd.nist.gov/vuln/detail/CVE-2018-13783>

⁶ <https://nvd.nist.gov/vuln/detail/CVE-2018-17968>

⁷ Ethereum corresponding cryptocurrency

2.2 Ethereum Virtual Machine (EVM)

Ethereum⁸ is a globally open decentralized blockchain framework that supports smart contracts, referred to as Ethereum Virtual Machine (EVM). Because EVM hosts and executes smart contracts, it is often referred to as the programmable blockchain. EVM contracts reside on the blockchain in a Turing complete bytecode language; however, they are implemented by developers using high-level languages such as Solidity or Vyper and then compiled to bytecode to be uploaded to the EVM. Users on the EVM can create new contracts, invoke methods in a contract, and transfer Ether. All of the transactions on EVM are recorded publicly and their sequence determines the state of each contract and the balance of each user. In order to ensure the correct execution of smart contracts, EVM relies on a large network of mutually untrusted peers (miners) to process the transactions. EVM also uses the Proof-of-Work (PoW) consensus protocol to ensure that a trustworthy third party (e.g., banks) is not needed to validate transactions, fostering trust among users to build a dependable transaction ledger. EVM gained remarkable popularity among blockchain users as EVM is the first framework that supports smart contracts to manage digital assets and build decentralized applications (DApps) (Khan and Namin 2020).

2.3 Ethereum Smart Contracts

A smart contract is a general-purpose digital program that can be deployed and executed on the blockchain. Ethereum smart contract is identified by a unique 160-bit hexadecimal string which is the contract address. It is written in a high-level language, either Solidity or Vyper. In this paper, we focus on Ethereum smart contracts written in Solidity because it is the most popular language in the EVM community and most of the deployed contracts on EVM are written using Solidity (Bhat and Vijayal 2017; Badawi and Jourdan 2020). A smart contract can call other accounts, as well as other contracts on the EVM. For example, it can call a function in another contract and send Ether to a user account. In EVM, internal transactions (i.e., calls from within a smart contract) do not create new transactions and are therefore not directly recorded on-chain.

2.4 Ethereum Gas System

In order to execute a smart contract, a user has to send a transaction (i.e., make a function call) to the target contract and pay a transaction fee that is measured in units of gas, referred to as the gas usage of a transaction. The transaction fee is derived from the contract's computational cost, i.e., the type and the number of executed instructions during runtime. Each executed instruction in the contract consumes an agreed-upon amount of gas. Instructions that need more computational resources cost more gas than instructions that need less computational resources. This helps secure the system against denial-of-service attacks and prevents flooding the network. Hence, the gas system in EVM has two main benefits: (1) it motivates developers to implement efficient applications and smart contracts and (2) it compensates miners who are validating transactions and executing the needed operations for their contributed computing resources. To pay for gas, the transaction fee equals the $gasprice * gascost$. The minimum unit of gas price is Wei (1 Ether = 10^{18} Wei). Therefore, Ether can be thought of as the fuel for operating Ethereum.

⁸ <https://ethereum.org/en/>

2.5 Solidity

Solidity⁹ is a domain-specific language (DSL) that is used to implement smart contracts on the Ethereum blockchain. It is the most widely used open-source programming language in implementing blockchain and smart contracts. Also, although it was originally designed to be used on Ethereum, it can be used in other blockchain platforms such as Hyperledger and Monax¹⁰. Solidity is statically typed, which requires specifying the type of all variables in the contract. It does not support any “undefined” or “null” values, and any newly defined variable has a default value based on its type. Smart contracts written in Solidity are organized in terms of subcontracts, interfaces, and libraries. They may contain state variables, functions, function modifiers, events, struct types, and enum types. Also, Solidity contracts can inherit from other contracts, and can call other contracts.

In Solidity there are two kinds of function calls. *Internal function calls*, which do not create an actual EVM call, and *External function calls*, which do. Due to this distinction, Solidity supports four types of visibility for functions and state variables:

- *External* functions can be called from other contracts using transactions as they are part of the contract interface. They can not be called internally (e.g., `externalFunction()` does not work, while `this.externalFunction()` works). State variables can not be external.
- *Public* functions can be called internally or using messages and they are part of the contract interface. For public state variables, an automatic getter function is generated by the Solidity compiler to avoid high gas cost when returning an entire array.
- *Internal* functions and state variables can only be accessed internally (i.e., from within the current contract or contracts deriving from it).
- *Private* functions and state variables are only visible for the contract they are defined in and not in derived contracts.

State variables can be declared using the keywords *constant* or *immutable*. Immutable variables can be assigned at construction time, while constant variables must be fixed at compile time. For the functions declaration, they can be declared as follows.

- *Pure Functions* promise not to read or modify the state. They can use the `revert()` and `require()` functions to revert potential state changes when an error occurs.
- *View Functions* promise not to modify the state such as writing to state variables.
- *Receive Ether Functions* can exist at most once in a contract. They cannot have any arguments and cannot return any value, and must be declared with external visibility and payable state mutability as in `receive() external payable{ ...}`. These functions are executed on plain Ether transfers (e.g., using `.send()` or `.transfer()`) and based on a call to the contract with empty *calldata*.
- *Fallback Functions* are similar to *Receive Ether* functions, as any contract can have at most one such function. It must have external visibility, and is executed on a call to the contract if none of the other functions match the given function signature or if no data was supplied at all and there is no receive function. The *fallback* function always receives data, but in order to also receive Ether it must be marked as *payable*.

Finally, if any function promises to receive Ether, it has to be declared as *payable*. An example of a Solidity contract is shown in Fig. 1. It shows a voting contract as explained in Solidity’s official documentation (Ethereum 2021).

⁹ <https://docs.soliditylang.org/>

¹⁰ <https://monax.io/>

```

pragma solidity >=0.7.0 <0.9.0;
contract Ballot {
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted;  // if true, that person already voted
        address delegate; // person delegated to
        uint vote;    // index of the voted proposal
    }
    struct Proposal {
        bytes32 name; // short name (up to 32 bytes)
        uint voteCount; // number of accumulated votes
    }

    address public chairperson;
    mapping(address => Voter) public voters;
    Proposal[] public proposals;
    constructor(bytes32[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
        for (uint i = 0; i < proposalNames.length; i++) {

            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }
    function giveRightToVote(address voter) external {
        require(
            msg.sender == chairperson,
            "Only chairperson can give right to vote."
        );
        require(
            !voters[voter].voted,
            "The voter already voted."
        );
        require(voters[voter].weight == 0);
        voters[voter].weight = 1;
    }
    function vote(uint proposal) external {
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "Has no right to vote");
        require(!sender.voted, "Already voted.");
        sender.voted = true;
        sender.vote = proposal;
        proposals[proposal].voteCount += sender.weight;
    }
}

```

Fig. 1 Voting Contract Example Written in Solidity (Ethereum 2021)

2.6 Common Vulnerability and Exposure (CVE) and National Vulnerability Database (NVD)

CVE is a list of publicly disclosed vulnerabilities and exposures that is maintained by MITRE¹¹. It feeds into the NVD¹², so both are synchronized at all times. NVD is a comprehensive repository with information about all publicly known software vulnerabilities and includes all public sources of vulnerabilities (e.g., alerts from security focus¹³). NVD also provides more information about the CVE lists' vulnerabilities, such as severity scores and patch availability. It also provides an easy mechanism to search for vulnerabilities using various variables. Both CVE and NVD are maintained by the US Federal Government. An example of a reported smart contract vulnerability in NVD is shown in Table 1. It also shows the impact metrics (Common Vulnerability Scoring System - CVSS), vulnerability types (Common Weakness Enumeration - CWE), applicability statements (Common Platform Enumeration - CPE), and other relevant meta-data. Each CVE in the list also has a unique identifier that shows the affected software product, sub-products, and various versions.

2.7 Common Weakness Enumeration (CWE)

CWE¹⁴ is a community-developed list of common software security weaknesses. It is considered as a comprehensive online dictionary of weaknesses that have been found in computer software. It also serves as a baseline for weakness identification, mitigation, and prevention efforts. Its primary purpose is to promote the effective use of tools to identify, find, and repair exposures in computer software before the programs are distributed to the public.

2.8 Smart Contract Weakness Classification Registry (SWC)

SWC is an implementation of the weakness classification scheme that is proposed in *Ethereum Improvement Proposals*¹⁵. It is also aligned with the terms and structures described in the CWE. Each SWC has an identifier (ID), weakness title, relationships, and related code samples list. In the relationships attribute, each SWC is mapped to the parent CWE and CWEs that are related.

2.9 Smart Contract Security

Due to the potential severe effects of exploited code weaknesses in smart contracts, security is an important research theme for blockchain platforms such as Ethereum and Hyperledger Fabric (Yamashita et al. 2019). Research on smart contract security follows a number of themes, e.g., principles and patterns of smart contract design (Philosophy 2019; Mavridou et al. 2019; Idelberger et al. 2016), secure development (Huang et al. 2019) and modeling

¹¹ <https://cve.mitre.org/cve/>

¹² <http://nvd.nist.gov/>

¹³ <https://www.securityfocus.com/>

¹⁴ <http://cwe.mitre.org>

¹⁵ <https://eips.ethereum.org/>

Table 1 An example of a vulnerability in NVD

CVE ID	CVE-2021-3006
NVD Published Date:	01/03/2021
Source:	CVE MITRE
Description	The breed function in the smart contract implementation for Farm in Seal Finance (Seal), an Ethereum token, lacks access control and thus allows price manipulation, as exploited in the wild in December 2020 and January 2021.
CVSS	7.5
Weakness Enumeration	CWE-863
CWE Name	Incorrect Authorization
Hyperlink	Link
Integrity	High
Impact score	3.6

¹⁴ <https://etherscan.io/address/0x33c2da7fd5b125e629b3950f3c38d7f721d7b30d>

(Idelberger et al. 2016), or verification, validation, and auditing Permenev et al. (2020); Kalra et al. (2018); Rameder (2021). Specifically focusing on vulnerabilities, there has been work studying the security of smart contracts after deployment as well as during execution (Huang et al. 2019), and suggesting best practices (Wan et al. 2021).

In our work, we focus on classifications of smart contract code weaknesses. We will discuss related work in this area more in the next section.

3 Related Work

Multiple studies that classify smart contract code weaknesses have been published since the first attack on Ethereum smart contracts in 2016 Daian (2016), e.g., Atzei et al. (2016), Alharby et al. (2018), Dingman et al. (2019), and Zhang et al. (2020). This section summarizes these studies in relation to our work.

3.1 Literature-Based Code Weaknesses Classification

Several studies on Ethereum smart contract weaknesses classify code weaknesses reported in blogs, academic literature, and white papers, i.e., Atzei et al. (2016); Alharby and Van Moorsel (2017); Huang et al. (2019); Sánchez (2018); Chen et al. (2020); Praitheshan et al. (2019); Dingman et al. (2019).

Atzei et al. (2016) propose a classification scheme comprised of three categories to classify code weaknesses in Ethereum smart contracts. In addition to blogs and academic literature, the authors also employ their own practical experience as a resource for their classification. Code weaknesses are classified into language-related issues, blockchain issues, and EVM bytecode issues. The classification is followed by a brief discussion on potential attacks that result in stealing money or causing other damage.

Alharby and Van Moorsel (2017) also use academic literature as the main source of data, classifying them into four main classes: codifying, security, privacy, and performance issues. Additionally, they discuss proposed solutions based on suggestions provided by smart

contract analysis tools. The proposed classification suffers from a significant overlap between categories. For example, codifying issues can lead to security and privacy issues, as in the case of the popular re-entrancy code weakness (classified as a security issue in the paper).

Huang et al. (2019) report a literature review of smart contract security from a software lifecycle perspective. The authors analyze blockchain features that may result in security issues in smart contracts and summarize popular smart contract code weaknesses based on four development phases, i.e., design, implementation, testing before deployment, and monitoring and analysis. Finally, they classified 10 code weaknesses into three broad categories (i.e., Solidity, blockchain and misunderstanding of common practices). Unfortunately, there is no explanation of how or on what basis these categories were designed.

Dingman et al. (2019) study well-known code weaknesses reported in white and gray literature and classify them according to National Institute of Standards and Technologies Bugs framework (NIST-BF)¹⁶ into security, functional, developmental, and operational code weaknesses. The results show that the majority of code weaknesses fall outside the scope of any category.

Similar to Dingman et al. (2019), Samreen and Alalfi (2021) survey and map eight popular smart contract code weaknesses in the literature to the NIST-BF. Their results show that only three of the studied eight code weaknesses could be matched with two NIST-BF classes. They also suggest a preventive technique per classified code weakness. Finally, a map between existing analysis tools and the eight code weaknesses are provided in the paper.

Praitheeshan et al. (2019) classify 16 smart contract code weaknesses reported in literature based on their internal mechanisms. The authors use three categories, i.e., blockchain, software security issues, and Ethereum and Solidity code weaknesses.

Finally, Chen et al. (2020) survey Ethereum system code weaknesses including smart contract code weaknesses reported in literature, classifying them according to two dimensions. First, they group code weaknesses into four-layer groups according to the location, i.e., on the application, data, consensus or network layer. Secondly, they group code weaknesses according to their cause into Ethereum design and implementation, smart contract programming, Solidity language and tool-chain, and human factors. This classification focuses on a few locations that a code weakness might occur at, while omitting others. For instance, the code weakness could be located in the source code of the smart contract itself or in its dependencies.

3.2 Repository-Based Code Weaknesses Classification

In addition to classifications that are based on published code weaknesses, two papers attempt classifications based on data extracted from public repositories such as StackExchange, i.e., Chen et al. (2020); Zhang et al. (2020).

Chen et al. (2020) collect smart contract code weaknesses from discussions available on Ethereum StackExchange, classifying them based on five high-level aspects according to their consequences, i.e., security, availability, performance, maintainability, and reusability. To evaluate if the selected code weaknesses are harmful, the authors conduct an online survey to collect feedback from practitioners. The proposed categories have the drawback that not all code weaknesses can be clearly placed in a single category, i.e., one code weakness could have various consequences.

Zhang et al. (2020) classify smart contracts code weaknesses from both literature and open projects on GitHub. The authors classify extracted code weaknesses into 9 categories

¹⁶ <https://samate.nist.gov/BF/>

based on an extension of IEEE Standard Classification for Software Anomalies¹⁷. Finally, they propose a four-category classification scheme for the impact of a code weakness, i.e., unwanted function executed, performance, security, and serviceability. Some of the proposed categories are overly general, such as the security category.

3.3 Tool-Based Code Weaknesses Detection

As a last category of related work, several publications study existing tools to detect vulnerabilities in smart contracts, and propose classifications based on the tools' capabilities, i.e., Khan and Namin (2020); Rameder (2021).

Khan and Namin (2020) provide an overview of current smart contract code weaknesses and testing tools in their work. In addition, they propose a code weakness taxonomy for smart contract code weaknesses. The proposed taxonomy consists of seven categories, i.e., inter-contractual code weaknesses, contractual code weaknesses, integer bugs, gas-related issues, transnational code weaknesses, deprecated code weaknesses, and randomization code weaknesses. The authors then provide a mapping between the surveyed code weaknesses and the available detection tools. Several of the categories in the proposed taxonomy overlap, e.g., a code weakness caused by an integer bug could lead to gas-related issues.

In a Master thesis, Rameder (2021) provides a comprehensive overview of state-of-the-art tools that analyze Ethereum smart contracts with an overview of known code weaknesses and the available classification schemes in the literature. The studied code weaknesses are classified into 10 novel categories, i.e., malicious environment, environment dependency/blockchain, exception & error handling disorders, denial of service, gas related issues, authentication, arithmetic bugs, bad coding quality, environment configuration, and deprecated code weaknesses.

3.4 Summary and Research Gap

In summary, various attempts to classify smart contract code weaknesses have been published, both on reported code weaknesses and by mining software repositories. Another line of research focuses on studying smart contract code weakness detection tools, classifying what code weaknesses they are able to detect. Overall, the proposed classifications are valuable to study smart contract code weaknesses from different dimensions, e.g., considering where in the network code weaknesses occur, or what the impacts of the code weaknesses are. However, existing classifications suffer from a number of flaws. First, several categorizations rely exclusively on code weaknesses reported in literature and might, therefore, provide a skewed image, e.g., Atzei et al. (2016). Secondly, we observe that many classifications mix different concerns or dimensions, such as consequences of exploiting a code weakness and the source of error in Rameder (2021). Third, existing categorizations propose general categories that do not allow for detailed reasoning, such as the privacy and security categories in Alharby and Van Moorsel (2017) and Zhang et al. (2020). Finally, they provide only a limited view on smart contract code weaknesses due to focusing on a single dimension, such as the consequences in Chen et al. (2020). Therefore, there is a need to provide a reference taxonomy that includes several dimensions such as root cause, impact, or scope (Vacca et al. 2021). The aim of this paper is to arrive at such a classification by integrating existing work and complementing

¹⁷ <https://standards.ieee.org/standard/1044-2009.html>

it with additional data from software repositories and well-known sources such as the CVE and SWC registries.

4 Research Method

This section presents the method we followed in this paper. Figure 2 shows an overview of the study method. Initially, we selected four data sources (Steps 1-4), i.e., Stack Overflow, GitHub, CVE, and CWE. We then collected data from the respective sources, cleaned and pre-processed it (Step 5). This was followed by manual labeling using open card sorting (Step 6). In Step 7, the resulting smart contract code weakness and impact classifications were mapped to literature-based classifications. We then defined critical code weaknesses and explained them (Step 8). As a final step, we analyzed the cleaned data quantitatively. A detailed description of each step follows in the remainder of this section.

4.1 Study setup

In this study, we aim to answer the following research questions (RQs):

- *RQ1. What categories of code weaknesses appear in smart contracts?*
Goal: To comprehensively categorize the code weaknesses that appear in Ethereum smart contracts. To study different dimensions of the problem and to map literature-based classifications on Ethereum smart contract code weaknesses and unify them in one thorough classification. A thorough classification scheme will make it possible to collect statistics on frequency, trends, and code weaknesses, as well as evaluate countermeasures.
- *RQ2. How do frequency distributions of smart contract code weakness categories compare across data sources?* Goal: To investigate if all data sources have the same frequency distributions of code weaknesses. If so, then we can rank code weakness categories from the most common to the least common. If not, we can reason about the skew or bias when

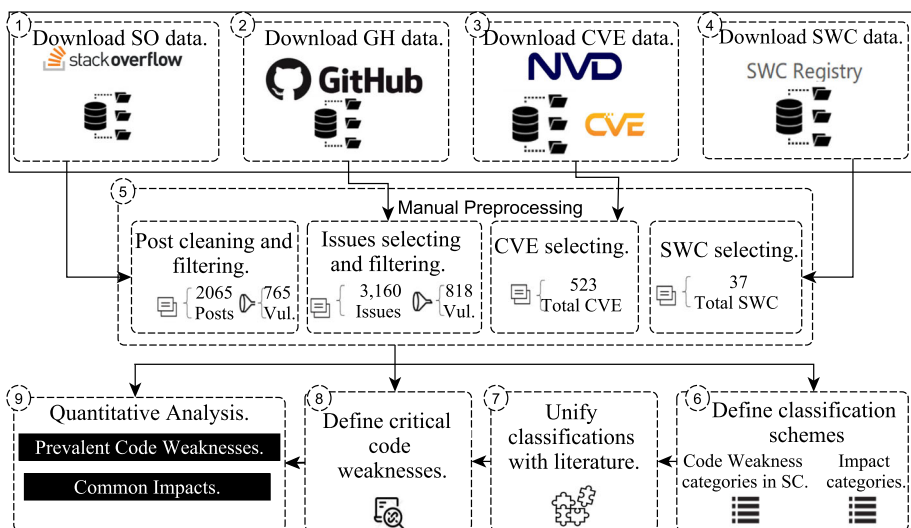


Fig. 2 Empirical Study Method

different sources are used. Further research can find solutions for the most common code weaknesses. More effort could be put to address the prevalent category before deploying the contract to the blockchain.

- *RQ3. What impact do the different categories of smart contract code weaknesses have?*
Goal: To investigate the impacts of smart contract code weakness on the product and the business. To define various dimensions of impacts classifications and to map literature-based impact classifications and propose a thorough impact classification of smart contract code weaknesses. More effort can be put to code weaknesses with critical impacts.

4.1.1 Data Sources

To answer the proposed research questions, we analyzed and studied smart contract code weaknesses from four primary sources. Two of these sources are widely used by developers (i.e., Github and StackOverflow), and two are very well-known publicly accessible sources (i.e., SWC and CVE) for reporting code weaknesses in Ethereum smart contracts and other software systems. Table 2 shows the final number of data records during and after data pre-processing. In all data sources, we extracted only records that were marked as vulnerable. That is, we did not judge whether or not smart contract code was indeed vulnerable. To assess the impact, we only used Stack Overflow, since posts included information on what went wrong in a smart contract. In GitHub, SWC and CVE, impact information was not readily available.

Data-source 1 We opted for extracting data from Stack Overflow, as it has successfully been leveraged in existing work on smart contracts (Ayman et al. 2019, 2020; Chen et al. 2020), and in general Software Engineering research Bajaj et al. (2014); Ponzanelli et al. (2014); Calefato et al. (2015); Chen and Xing (2016); Ahasanuzzaman et al. (2016).

We used Stack Overflow posts to study code weaknesses in smart contracts. To do so, we extracted Q&A posts tagged with “smart contract”, “Solidity”, and “Ethereum”, posted between January 2015 and April 2021. We discarded posts with the “Ethereum” tag, but without the “Solidity” or “smart contract” tags. To retrieve the related information for each post, we used Scrapy¹⁸, an open source Python library that facilitates Web crawling. For each of the 2065 extracted posts, we extracted the post title, URL with the description of the code weakness, related tags, post time and accepted answer time.

Data-source 2 We used GitHub as the second data source for our study, as it is the most popular social coding platform (Cosentino 2017). Moreover, many studies on Ethereum smart contracts and smart contract analysis tools have been published reporting findings based on data published in GitHub open-source projects (e.g., Durieux et al. (2020)). We studied code weaknesses reported in open source projects that have Ethereum smart contracts written in Solidity. We used the keywords “smart contract”, “Solidity”, and “Ethereum” to search for these projects. We also collected the code weaknesses found in the closed reports (i.e., the closed issues that have the keyword “fix”) in GitHub.

We also studied the official Ethereum GitHub repository. Figure 3 shows an example of a reported smart contract code weakness in GitHub.

Data-source 3 We used the NVD search interface to collect and extract all reported CVEs on smart contracts and their CVSS until April 2021. We searched with “smart contract”, “Solidity”, and “Ethereum”. Then, we manually extracted the reported CVEs that are related to smart contracts.

¹⁸ <https://scrapy.org>

Table 2 Summary of the sampled code weaknesses in the selected data sources

Data Source	Stack Overflow	GitHub	CVE	SWC
# of collected data	2065	3160	523	37
# of data records after preprocessing steps	1490	1160	523	37
Final # of Code weaknesses	765	818	523	37

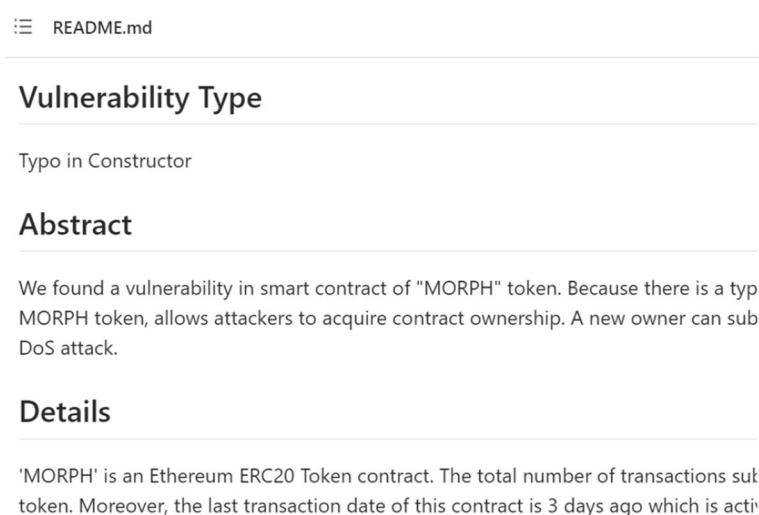
Data-source 4 We extracted all the reported code weaknesses in SWC until April 2021. For each code weakness, we extracted the ID, title, relationships, and test cases.

The collected data records were stored in Excel sheets (one for each data source) for later pre-processing and cleaning.

4.1.2 Data Cleaning and Pre-Processing

After the initial extraction, we applied several filtering steps to obtain a clean dataset. First, we removed posts with duplicate titles or marked as *[duplicate]* keyword in Stack Overflow, and we manually inspected the title and the body of the question and decided if the post actually discussed smart contracts in Ethereum/Solidity. Finally, we removed vague, ambiguous, incomplete, and overly broad posts. As an indication for such posts, we considered the amount of negative votes and/or negative feedback. The threshold of the negative votes was set to two negative votes or two negative comments. Finally, we extracted the code of the smart contract in each post for further analysis.

In order to pre-process the collected GitHub data, we removed the duplicates based on the description URL of the code weakness. If two records in the database with the same code weakness description URL and type, then we considered these two records as duplicate code weakness and removed one of them. For further analysis, we also extracted the code of the smart contract that contained the code weakness. We also manually double-checked data

**Fig. 3** Code weakness Example from GitHub

collected from both the NVD database and SWC registry for duplicates. After this stage, we had a clean dataset with records from the four data sources.

Table 2 shows a summary of the collected dataset after applying the aforementioned cleaning and pre-processing steps.

4.2 Data Analysis and Classification Categories

To analyze and label the cleaned dataset, we manually inspected each record and read the description of each code weakness. Then, to define the categories of code weaknesses in smart contracts, two experts (i.e., a doctoral student¹⁹ with a software engineering background focusing on smart contract security, and a cybersecurity expert with experience in cryptocurrencies, smart contracts, and security incidents) together used open card sorting (Spencer 2009) to propose a classification scheme based on the cleaned data. We used an open card sorting approach, since code weakness categories in the selected data were unknown and could potentially differ from existing classifications. Furthermore, open card sorting allowed us to define categories that are more specific compared to existing ones, and potentially more useful. Finally, the approach ensured the classification is grounded in real-world smart contracts and developer discussions. Open card sorting is appropriate in this context, as designing the different code weakness categories is a creative, qualitative task. Disagreements between the two experts were used to discuss potential differences and come up with a categorization of higher quality. Data sources were not weighted anyhow, as all records were taken into account.

Card Sorting for Taxonomy Construction Card sorting, a prevalent user-centered design technique, involves prompting participants to categorize a set of cards into meaningful groups, thereby revealing their implicit item grouping tendencies (Wood and Wood 2008). Widely utilized in the construction of taxonomies, card sorting provides valuable insights into the process of information categorization (Spencer 2009). It allows researchers to gain a deeper understanding of the inherent structure and relationships within a given set of items. Each card or item can be an object, a picture, an attribute name Maiden (year); Rugg and McGeorge (1997). There are different types of card sorting techniques. In open card sorting, participants have the freedom to generate their own categories and allocate items without predetermined labels or constraints (Sakai and Aerts 2015). Conversely, closed card sorting provides participants with predefined categories into which they sort the items (Sakai and Aerts 2015). Hybrid card sorting combines elements of both open and closed sorting, enabling participants to create new categories while also offering a predefined set of categories as a reference Nawaz (2012). The selection of a specific card sorting technique depends on the research objectives, contextual factors, and available resources.

Open card sorting settings participants, and materials The open card sorting exercise was conducted over approximately 6 months in total, with the two experts involved throughout. The experts were selected based on availability and their experience in relevant areas. As materials, we used a list of the selected data sources and an introduction to the open card sorting exercise with the goal of the exercise. We also provided a description of what information each card should contain with a brief description of the information extracted from each data source as mentioned before. The two experts then discussed the sorting process.

For each record in the cleaned dataset, we created a card containing the information of the code weakness as collected from the original data source and a unique ID. A summary

¹⁹ The first author of this paper.

of the information included in the cards for each data source is shown in Table 3. All cards were stored on two separate computers, as to avoid the experts influencing each other.

Open Card Sorting Procedure First, the two experts labeled a random 10% of the data, and measured inter-rater agreement after independent labeling using Cohen's Kappa coefficient. The Kappa value between the two experts was 0.60, showing moderate agreement Viera et al. (2005).

After this session, the two experts discussed the categories in a clarification session, where categories deemed overly broad were discarded. Some categories were merged, and the two experts agreed on the naming of the resulted fine-grained classes. An example of two merged categories are “access control” and “authentication”, where the experts decided to use “authentication and authorization” as a common category. The categories were generally labeled based on the causes mentioned in the descriptions in each card. That is, we analyzed the question and the answer information on Stack Overflow, code weakness abstract and details on GitHub, CVE description in the NVD database, and the SWC's description.

Disagreements among the experts were resolved through discussions. Labeling took two to three hours every weekday. For instance, the experts disagreed whether or not “structural data flow” should be included in the category named “logic”. Ultimately, both categories were kept, as the causes for the weaknesses differed in the data. Common sources of disagreement were cases where causes of a weakness were not stated clearly in the card. In such cases, the experts consulted the Solidity code connected to the reported weakness and discussed the cause until reaching an agreement. After completing the first 10% of the data, the existing categories were used as base categories for the remainder of the data.

The two experts then followed the same procedure, with one open card sorting session per working day. Clarification discussions were also repeated after 20% and 40% of the posts were labeled (with Kappa values of 0.75 and 0.82 respectively). Similarly to the first classification session, these sessions were used to clarify major disagreements. Finally, calculations of the Kappa coefficient at 60%, 80%, and 100% resulted in values of > 0.90 , indicating perfect agreement.

The same approach was followed to label the impact of each code weakness in the cleaned dataset. The Kappa value was also > 0.90 in the final discussion of the impact labeling.

Table 3 An overview of the information included in the card for each data source

Data Source	Card information
Stack Overflow	Tag, title, description, expert 1 label, expert 2 label, and agreement.
GitHub	ID, description URL, expert 1 label, expert 2 label, and agreement.
CVE	CVEName, description, CWE, expert 1 label, expert 2 label, and agreement.
SWCs	Title, description, CWE, expert 1 label, expert 2 label, and agreement.

4.3 Unifying Classification Schemes

To unify existing code weakness classification schemes in the literature, we gathered all the categories proposed in the literature into an Excel sheet. The first and the second authors then discussed each category in relation to the categories proposed by us. We subsequently defined three dimensions of the problem (i.e., the code weakness's source of error; the location of the code weakness in the network level; the behavior and consequences arising from an exploit of the code weakness). Afterwards, we mapped existing classifications to the defined dimensions, as illustrated in Fig. 4. We name these code weakness dimensions *V-D*. The error source dimension (V-D1) describes the main cause that, when triggered, can result in executing the code weakness, such as the logic of the contract and the data initialization. V-D2 is the network dimension, which indicates at which layer on the network the code weakness occurs. Ethereum network consists from several layers; (1) application layer (i.e., the code weakness can be in the user account), (2) blockchain data layer (i.e., the weakness can be in the blockchain data structure), or (3) consensus layer (i.e., the weakness can be at the consensus protocols), etc Chen et al. (2020). Finally, V-D3 describes the resulting behavior and consequences of the code weakness, which means the result of executing the code weakness.

We followed the same approach to devise a thorough classification scheme for the impacts of smart contract code weaknesses. We defined two dimensions (i.e., impact on the software product, and impact on business factors) as shown in Fig. 5. We name these impact dimensions *I-D*. I-D1 is the impact of the code weakness on the software product itself (i.e. the smart contract as a digital program) and its resulting behavior. For instance, software engineers can describe the poor performance of a contract. As such, I-D1 considers the behavior of smart contracts from a software engineering perspective. I-D2 describes the impact of the code weakness on business factors and the contract owners, e.g., losing money or important information of the business owners.

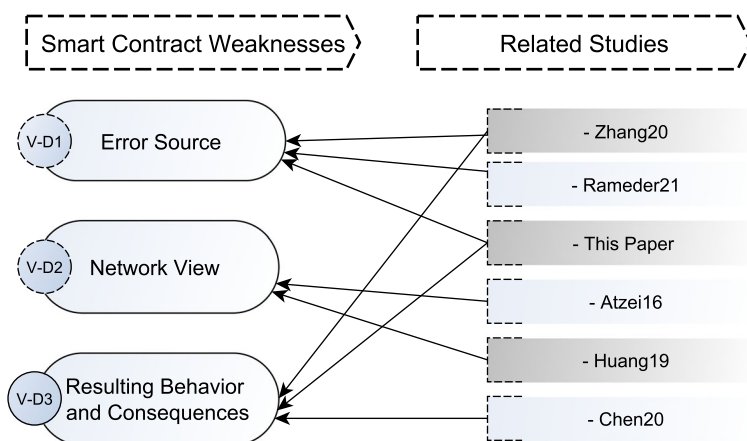


Fig. 4 Dimensions of Smart Contract Code Weaknesses (*V-D*)

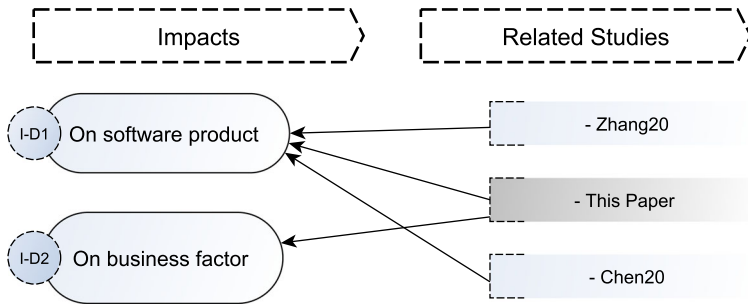


Fig. 5 Dimensions of Smart Contract Code Weakness Impacts (*I-D*)

5 Results

In this section, we present the results of our study, and answer the proposed RQs. Our objective is to unify the existing classification schemes and define the causes, impacts, and recurrences of smart contract code weaknesses. Moreover, we propose thorough classification schemes for both the impacts and categories of smart contract weaknesses.

Smart contract code weaknesses are found in more than half (i.e., 66.7%) of the cleaned data from all the four data sources (Stack Overflow, GitHub, CVE and SWC).

5.1 What categories of code weaknesses appear in smart contracts? (RQ1)

To answer RQ1, we followed the analysis approach in Section 4.2, then mapped the result of our classification to other classification schemes as explained in Section 4.3. We classified the 2143 extracted weaknesses into 47 unique smart contract weaknesses, grouped into 11 categories. Within our classification scheme, we mapped the existing literature classification schemes. We were able to define 47 critical code weaknesses from the data we collected. In this work, 21 novel code weaknesses are introduced to the body of knowledge.

Table 4 briefly summarizes our proposed categories of smart contract code weaknesses and their definitions.

Table 5 maps literature-based classification schemes of smart contract code weaknesses with our own. The categories proposed in the literature are listed in the rows, while our categories are listed in the columns of the table. The table covers all the three dimensions of *V-D* discussed in Section 4.3. As can be seen, some broad categories listed in literature essentially cover all of our classification categories.

Most literature-based classification schemes for smart contract code weaknesses include broad categories, such as security and availability.

The following subsections present the key findings related to the defined categories. For each category, we define and explain the most critical code weaknesses as agreed by the two raters. We give examples for some weaknesses, which are directly taken from our dataset.

Table 4 Classification Scheme of Code Weaknesses in Smart Contracts

Category Name	Description	Short
Language Specific Coding	Syntax mistakes in implementing Solidity contracts that are not captured by the Solidity compiler can introduce unexpected behavior or damage the contract.	CV
Data Initialization and Validation	The input data type to the contract or the fields' data type in the contract are not initialized correctly. Also includes the data passed from/sent to other contracts. Predictable resources is a subcategory of data initialization and validation. Weaknesses resulting from using expected values in state variables or functions. These code weaknesses can allow malicious minors to take advantage of the code weaknesses and control the contract.	DV DV-PR
Structural Sequence & Control	Problem with the structure of the contract control flow. Specifically, a result of incorrect control flow structure such as require, if-else conditions, assert, and loop structures.	SV-SC
Structural Data Flow	Problems with the structure of the contract data flow. The main difference to <i>Data</i> is that <i>Data</i> originates in the data fields and input parameters to the subcontracts or the contract methods. Instead, in this category, changing data fields in a wrong way during and after the execution of the contract leads to issues.	SV-DF
Logic	Inconsistency between the intention of the programmer and the coded contract, and not one of the other categories.	LV
Timing & Optimization	Performance and timing issues that can affect execution time and results in abnormal responsiveness under a normal workload.	TV
Compatibility	Required software and packages are not compatible with the available resources (e.g., operating system, CPU architecture).	CoV
Deployment & Configurations	Weaknesses during deployment of implemented contracts on the Ethereum blockchain.	DL
Authentication & Authorization	Code weaknesses allow malicious actors to take control over the contract.	SV
Dependency & Upgradability	Upgrades in a smart contract breaking the dependencies in the new contract.	UV
Interface	Code weaknesses in the interface of smart contracts, e.g., when the contract is functioning correctly, but the interface is showing a wrong output that contradicts the contract's execution logs and transaction logs.	IB

Table 5 Mapping Literature-based Classifications to V-D

Category	CV	DV	SV	LV	TV	CoV	DL	SV	UV	IB
Codifying Alharby and Van Moorsel (2017)	C*			C*						
Data* Zhang et al. (2020)	*C	*C								
Description* Zhang et al. (2020)	*C									
Interaction* Zhang et al. (2020)	*C	*C		*C						
Interface* Zhang et al. (2020)										=
Logic* Zhang et al. (2020)				=						
Standard* Zhang et al. (2020)	*C									
Authentication* Rameder (2021)								=		
Arithmetic* Rameder (2021)				=						
Bad Coding Quality* Rameder (2021)	=									
Environment Configuration* Rameder (2021)							=			
Deprecated* Rameder (2021)	*C									
Solidity* Atzei et al. (2016)	*C									
EVM* Atzei et al. (2016)		C*	C*	C*	C*	C*	C*	C*	C*	
Blockchain* Atzei et al. (2016)						C*				
Security* Alharby and Van Moorsel (2017)	C*	C*	C*	C*	C*	C*	C*	C*	C*	C*
Privacy* Alharby and Van Moorsel (2017)	C*	C*		C*						
Performance* Alharby and Van Moorsel (2017)					=					
Availability* Chen et al. (2020)	C*	C*	C*	C*	C*	C*	C*	C*	C*	C*
Maintainability* Chen et al. (2020)	C*		C*	C*		C*			C*	
Reusability* Chen et al. (2020)	C*	C*	C*	C*			C*	C*	C*	C*

C* indicates that the category in the V-D classification is a subset of the corresponding category in the literature marked by *. *C means the category in the literature is a subset of a proposed category in V-D. Finally, = means the categories are identical

For the full list of code weakness definitions, we refer to our published dataset (Soud et al. 2021).

5.1.1 Language Specific Coding Weaknesses

In this category, smart contract code weaknesses result from language-based errors not captured by the compiler. The source of error of this category can be in the language pre-defined functions, events, libraries, and/or language standards.

We define 14 Language Specific Coding weaknesses that can result in undesirable state in a smart contract or can be used by attackers in their favor. Next, we show a sample of these weaknesses.

Insufficient compiler version or pragma version — CV#1 A so-called version *pragma* should be included in the source code of smart contract to reject compiling the contract using incompatible compiler versions. When using a version *pragma* in the contract which is later than the selected compiler, this may introduce incompatible changes and lead to code weaknesses in compiled smart contract code. Moreover, future compiler versions may handle language constructs in a way that introduces unclear changes affecting the behavior of the contract as shown in Listing 1.

```
1 pragma solidity ^0.6.3; // weakness
2 pragma solidity 0.6.3;
```

Listing 1 Version pragma

Fallback function not payable — CV#2 Smart contracts written in Solidity versions 0.6.0+ should have the fallback function split up into a *receive()* and a *fallback()* function (i.e., a new fallback function that is defined using the *fallback* keyword and a *receive* ether function defined using the *receive* keyword). If present, the *receive* function is called whenever no parameters are provided in the call. The *receive* function is implicitly payable. The new *fallback* function is called when no other function matches. However, if *fallback()* is not payable and *receive()* does not exist, transactions not matching any other function which send ether will revert and result in an undesirable state in the contract.

Fallback function does not exist — CV#3 In addition to CV#2, when sending Ether from a contract to another contract without calling any of the receiving contract's functions, sending the Ether will fail if the receiver contract has no fallback function. Thus, a payable fallback function should be added to the receiver before deployment. Otherwise, there is no way to receive the Ether unless the sender has previous knowledge of the exact functions of the receiving contract, which is not usually the case.

Violating splitting Fallback function — CV#4 Smart contracts written in Solidity versions 0.6.0+ should have the fallback function split up to *receive()* and *fallback()*. If present, the *receive* function is called whenever the call data is empty. The *receive* function is implicitly payable. The new *fallback* function is called when no other function matches. The fallback function in Listing 2 can be payable or not, however, if it is not payable then transactions not matching any other function which send value will revert.

```
1 contract payment{
2     mapping(address => uint) _balance;
3     fallback() payable external {
4         _balance[msg.sender] += msg.value;
5     }
6 }
```

Listing 2 Fallback function

Violating modifier definition — CV#5 Solidity provides modifiers that are used to change the behavior of functions in a declarative way. They can be used to enforce pre/post-conditions on the execution of a function. The `_` operator should be used in defining a modifier, and starting from Solidity version 0.4.0+ a semicolon should be added after the `_` operator. The operator represents the actual code of the function that is being modified. Thus, the code for the function being modified is inserted where the `_` is placed in the modifier. Missing the `_` operator might generate unwanted results. For example, in Listing 3, line 8, every time `transferOwnership` is invoked, the `onlyOwner` modifier will get into play first. If the owner invokes it, then the control flow will reach the `_` operator, so the `transferOwnership` statements will be executed. Otherwise, the execution will just throw, revert, and exit.

```

1 contract owned {
2     address public owner;
3     function owned() {
4         owner = msg.sender;
5     }
6     modifier onlyOwner {
7         if (msg.sender != owner) throw;
8         _;
9     }
10    function transferOwnership(address newOwner) onlyOwner {
11        owner = newOwner;
12    }
13 }
```

Listing 3 Violating *modifier* definition

Manipulated language standard — CV#6 Ethereum has adopted many standards to guarantee the composability of smart contracts and Dapps. Those standards are in Ethereum’s official EIPs and include token ²⁰ standards. For example, ERC-20 is a token technical standard that allows developers to implement tokens of cryptocurrencies. It contains nine unique functions and two events to guarantee the possibility of exchanging tokens based on ERC-20 with other ERC-20 tokens. Any modification on the function name, parameter types, and the return value in the standard might change its functionality and leave the developer believing it is the same as ERC-20. The implementation of ERC-20 in any contract shall strictly be the same as in the standard template.

Violating call-stack depth limit — CV#7 In Ethereum, the call-stack has a hard limit of 1024 frames. Each time the contract calls an external contract, the call-stack depth of transaction increases by one. Thus, when the number of calls exceeds the limit of the call-stack, an exception is thrown and the call is aborted by Solidity. Moreover, Solidity does not support exceptions in low-level external calls. Therefore, a malicious actor can recursively call a contract 1023 times, then call a victim contract to reach the call-stack depth limit. This will fail any subsequent call made by the victim contract without the victim contract owner being aware of the attack. Recently, EIP 150²¹ makes it impossible to reach stack depths of 1024, effectively eliminating call depth attacks

Insufficient Address split — CV#8 Starting from Solidity 0.5.0+, the address should be split to address and address payable, where only address payable provides the transfer function. Otherwise, sending tokens to “unpayable” addresses will be reverted. Moreover, there is no way to convert an address to address payable.

Mixing pure and view — CV#9 In Solidity, *pure* and *view* are function modifiers that describe how the logic in that function will interact with the contract’s state. Functions that

²⁰ An Ethereum token can represent anything, including lottery tickets, financial assets, a fiat currency like USD, an ounce of gold, etc.

²¹ <https://github.com/ethereum/EIPs/blob/master/EIPS/eip-150.md>

are declared *view* promise not to modify the state, while functions that are declared *pure* promise not to read or write the state. By using no specifier, the state can be read as well as modified. Developers can mix these two modifiers by replacing *view* with *pure* or missing any of these modifiers, resulting in unexpected state changes or incorrect reads from the state.

Using of *balance* as attribute to the contract — CV#10 One of the features of Solidity is that contracts inherit all members from *Address*, meaning that the keyword *this* is the pointer to the current instance of the type derived from *Address*. In other words, if the developer wants to access members of the address type (e.g. the *balance*) of the current contract instance, then the developer should use *this* and should use *balance* as an attribute of the address type, not the contract as shown in Listing 4. We noticed a confusion in using *balance* and other address attributes as if they are attributes of the contract.

```

1 function getSummary() public view returns (
2     uint, uint, uint, uint, address
3 ) {
4     return (
5         minimumContribution,
6         this.balance, //incorrect
7         address(this).balance, // correct
8         requests.length,
9         approversCount,
10        manager
11    );
12 }
```

Listing 4 Using of *balance* as attribute to the contract

Unsafe delegatecall (code injection)— CV#11 A special variant of a message call in Solidity is *delegatecall*. With this feature, the contract can be executed in the callee's context, while *msg.sender* and *msg.value* remain unchanged. Consequently, it allows an external contract to modify the storage of the calling contract. This can be exploited by a malicious caller to manipulate the caller's contract state variables and take full control over the balance.

Variable shadowing — CV#12 Solidity supports ambiguous naming when inheritance is used. For instance, contract Alpha with a variable *V* could inherit contract Beta that also has a state variable *V* defined. Consequently, there would be two versions of *V*, one accessed from contract Alpha and the other from contract Beta. In complex contract systems, this condition might go undetected and ultimately cause security issues. Also, this can also occur at the contract level (e.g., a contract with more than one definition at the contract and function level).

Deprecated Solidity code — CV#13 As Solidity evolves, several of its functions and operators are deprecated. Making use of them leads to poor code quality. It is strongly discouraged to use deprecated Solidity language code with new major versions of the Solidity compiler, since it can cause unwanted behavior.

Experimental Language Features — CV#14 Similar to CV #13, it is strongly discouraged to use experimental Solidity language features since it can cause undesired behavior.

5.1.2 Data Weaknesses

Most of the data weaknesses result from the use of wrong or insufficient data types, or passing wrong data formats to arguments without knowing the exact required type. Moreover,

organizing the memory and storage in Solidity is the responsibility of programmers, which many developers are not used to do.

Violating explicit data location — DV#1 For Solidity versions 0.5.0+, an explicit data location for all variables of type struct, array, or mapping is mandatory. This also applies to function parameters and return variables. For instance, *calldata* is a special data location that contains the function arguments, which is only available for external function call parameters. If *calldata* is not included in the initialization, it results in unexpected values as shown in Listing 5.

```

1 contract StructExample {
2
3     struct SomeStruct {
4         int someNumber;
5         string someString;
6     }
7     SomeStruct[] someStructs;
8
9     function addSomeStruct() {
10         SomeStruct storage someStruct = SomeStruct(123, "test")
11         ;// insufficient use
12         SomeStruct memory someStruct = SomeStruct(123, "test")
13         ;// correct
14         someStructs.push(someStruct);
15     }
16 }

```

Listing 5 Using of *torage* instead of *memory*

Using of storage instead of memory — DV#2 In addition to *calldata*, Solidity provides two more reference types to comprise structs, arrays, and mappings called *storage* and *memory*. The Solidity contract can use any amount of memory (based on the amount of Ether that the contract owns and can pay for) during execution. However, when execution stops, the entire content of the memory is wiped, and the next execution will start fresh. The *storage* is persisted into the blockchain itself, so the next time the contract executes, it has access to all the data it previously stored in its storage area. Confusing storage and memory can result in data loss.

Violating array indexing — DV#3 Developers are making numerous mistakes when initializing and accessing arrays in Solidity. Most of the time, discovering these violations is not easy, especially if there is no syntax error or an error that can be detected by the compiler. This can result in returning incorrect values. A clear violation of array indexing is shown in Listing 6, line 9, where the developer is trying to access a single element in a 3-dimensional array, but only provides two sets of square brackets. Therefore, the developer is returning an array instead of a single Object.

```

1 contract Game {
2     struct User{
3         address owner; }
4     User[][10][10] public gameBoard;
5     User memory mover = gameBoard[_fromX][_fromY][0];
6     function addUser (uint _x, uint _y) public {
7         gameBoard[_x][_y].push(User(msg.sender, 10, 5, 5, 5,
8         5));}
9     function moveUser (uint _fromX, uint _fromY) public
10     {

```

```

9      User memory mover = gameBoard[_fromX][_fromY]; //
      incorrect access
10     if (mover.owner != msg.sender) return; })

```

Listing 6 Violating arrays indexing

Hard-coded address — DV#4 An existing bad practice is the use of hard-coded addresses in smart contract code, as shown in Listing 7. Any incorrect or missing digit in the address may result in losing Ether, in case Ether is sent to that wrong address, or in unexpected outcomes. This code weakness is also known as “Transfer to orphan address” Atzei et al. (2016).

```

1 address recieveraddress ;
2 function initializeAddress1 () {
3     recieveraddress = 0x98081c...8e5ace; //hardcoded address}

```

Listing 7 Hardcoded address

Improper data validation — DV#5 It is necessary to validate input from untrusted sources, such as external libraries or contracts before integrating it into any contract logic.

Unintentional Write to arbitrary storage location — DV#6 Because Solidity storage is not dynamically allocated, it can lead to unpredictable behavior, unauthorized data access, and other code weaknesses, especially if the data location of data types like structs, mappings, and arrays is not clarified and allowed to overwrite entries of other data structures.

5.1.3 Predictable Data Values and Resources

We encountered several issues in Solidity smart contracts that relate to values that can be guessed even though they are intended to serve as an element of randomness.

Timestamp dependency — DV-PR#7 To keep contracts safe from malicious actors, developers should avoid using the block variables as a source of randomness or as part of triggering conditions for executing significant operations in their contracts, such as transferring Ether. When submitting blocks, miners determine the value of block variables such as block.timestamp, block.coinbase, and block.difficulty. Thus, these values can affect the contract’s outcome and can be used to benefit the attacker. For example, Listing 8 shows an insecure lottery contract in which block.timestamp is used as a source of entropy.

```

1 function setWinner() public {
2
3     bytes32 hash = keccak256(abi.encode(block.timestamp));
4     bytes4[2] memory x0 = [bytes4(0), 0];
5     assembly {
6         mstore(x0, hash)
7         mstore(add(x0, 4), hash)
8     }

```

Listing 8 Insecure lottery contract using block variables

Blockhash dependency — DV-PR#8 Using blockhash has the same risks as block.timestamp in DV-PR #7, especially when used in critical operations such as Ether transfer. It can lead to serious attacks as malicious miners can tamper with the blockhash and take full control over the contract.

Bad random number generation— DV-PR#9 Using random numbers is not avoidable in some smart contracts, e.g., games or lotteries. It is important that the randomness is not based on global blockchain variables, as that leaves the contract open to manipulation by malicious miners similar to DV-PR#7 and DV-PR#8.

5.1.4 Sequence and Control Weaknesses

This category of code weaknesses is corresponding to incorrect control structure and loop control statements. These can be exploited and help the attacker to steal money in the contract. Moreover, they can also result in losing all the money in the contract without even being attacked, just because of code weaknesses in these structures.

Using assert instead of require — SV-SC#1 The *assert* statement should be only used for conditions that indicate you have an internal code weakness in the contract code. The *require* statement should be used to check valid conditions (e.g., state variables conditions are met, validate input, and validate return value from external contracts). A valid code with correct functions should never fail *assert* conditions. Otherwise, there is a code weakness in the contract and something unexpected has happened. In smart contracts, *assert* can consume all the gas in the contract, as shown in Listing 9. If the example is tested with `run(8)`, the function runs successfully and 1860 gas will be consumed based on the cost of the function and the loop iterations²². If it is tested with `run(15)`, then the function passes *assert*, fails at *require* and the first loop only will be executed and consume 1049 gas. Finally, testing the same example with `run(25)` causes the function to fail the *assert* statement. The execution continues and thus iterates 25 times through the loop, resulting in a very high cost of gas.

```

1 contract Test {
2     function run(uint8 i) public pure {
3         uint8 total = 0;
4         for (uint8 j = 0; j < 10; j++)
5             total += j;
6         assert (i < 20);
7         require (i < 10);
8         for (j = 0; j < 10; j++)
9             total += j;
10    }

```

Listing 9 Using *assert* instead of *require*

5.1.5 Data Flow Weaknesses

These are code weaknesses in the data flow of smart contracts, so that data fields are changing unexpectedly or incorrectly. We defined two code weaknesses that belong to this category.

Updating storage in fallback functions— SV-DF#1 Upon receiving Ether without a function being called, either the `receive` Ether or the fallback function is executed. If the contract does neither have a `receive` nor a fallback function, the Ether will be rejected by throwing an exception. During the execution of one of these functions, the contract can only rely on the passed gas (i.e., 2300 gas) being available to it at that time. This stipend is not enough to modify storage. However, we found that developers sometimes are updating state variables, trying to write to the storage in the fallback functions. Updating the variables with such a gas limit will fail. If the data flow of the contract depends on the failed state variables, this results in an incorrect data flow and in unexpected outcomes.

Arithmetic operation/overflow and underflow — SV-DF#2 An overflow can happen as a result of an arithmetic operation or calculation that falls outside the range of a Solidity data type, resulting in unwanted behavior or unauthorized manipulation of the contract balance.

²² The actual gas costs are stated in the Solidity documentation and depend on numerous factors, such as the executed functions and the used data types.

Underflow happens when an arithmetic operation reaches the minimum size of a type. This is a data flow code weakness, as the code of the contract does not perform correct validation on the numeric inputs and the calculations. In addition, the Solidity compiler does not enforce detecting integer overflow/underflow. An example is shown in Listing 10, where the computation of *mask* overflows at $x \geq 248$.

```

1 uint256 public MAXUINT256 = 2*256 - 1;
2 for (uint256 x = 0; x < 255; x++) {
3     var mask = MAXUINT256 * (2 ** x);
4     var key = signature & bytes32(mask);

```

Listing 10 Arithmetic Operation/calculation overflow

5.1.6 Logic Weaknesses

This category reflects inconsistencies with the contract and the programmer's intention, which is usually mentioned in the question information. These issues relate to a number of reasons, e.g., vague developer intentions, misunderstanding of language components, incorrect usage of Math, and incorrect gas predictions.

Greedy contract—LV#1 This code weakness occurs when implementing a contract logic that is only locking Ether balance all the time because of its inability to access the external library contract to transfer Ether. For instance, the contract logic may only accept transferring money based on a specific value in the code, which happens to be unreachable due to incorrect logic. In this case, the Ether will be locked in the deployed contract forever. In the example of Listing 11, the function *refundMoney()*, line 8, does not decrease the *weiRaised* value, meaning that once starting a refund, the developer can no longer use *forwardAllRaisedFunds()* to drain the contract. This code weakness would be triggered even in the regular course of action and Ether in this contract is stuck. It can receive any funds but the received funds can never be retrieved.

```

1 contract SwordCrowdsale is Ownable {
2     //amount of raised money in wei
3     uint256 public weiRaised;
4     bool public isSoftCapHit = false;
5     //send ether to the fund collection wallet
6     function forwardAllRaisedFunds() internal {
7         wallet.transfer(weiRaised);
8     }
9     function refundMoney(address _address) onlyOwner public {
10        uint amount = contributorList[_address].
11        contributionAmount;
12        if (amount > 0 && _address.send(amount)) {
13            //user got money back
14            contributorList[_address].contributionAmount = 0;
15            contributorList[_address].tokensIssued = 0;
16        }
17    }

```

Listing 11 Greedy Contract

Transaction order dependency — LV#2 The code weakness arises when a contract's logic is dependent on the order in which transactions are executed and processed in a block. It is a type of race condition inherent to Blockchains. By manipulating the order of transaction processing, malicious miners can take advantage of the contract and benefit from it. Therefore, the logic of the contract should not rely on the transaction order.

Call to the unknown — LV#3 This code weakness arises when a function unexpectedly invokes the fallback function of the recipient. Consequently, malicious code can be introduced. For example, the unknown call could trigger the recipient's fallback function, allowing malicious code to execute. Also, this can be done via direct call, delegatcall, send, or only call functions. In the MultiSig Wallet Attack²³, an attacker exploited this code weakness to steal 30M USD from the Parity Wallet. Another example is shown in Listing 12. In which, pong function uses a direct call to invoke Alice's ping. However, if the interface of contract Alice by mistake was mistyped by declaring the parameter as *int* instead of *unit* and Alice has no function with *int* type, then the call to ping results in calling Alice's fallback function.

```
1 contract Alice {function ping(uint) returns (uint)}
2 contract Bob {function pong(Alice c){c.ping(42);}}
```

Listing 12 Call to the unknown Atzei et al. (2016)

DoS by external contract — LV#4 It is possible for external calls to fail and throw exceptions or revert the transaction. Inefficient management of these calls in the contract logic can lead to critical code weaknesses, such as a Denial of Service (DoS) or loss of funds.

5.1.7 Timing and Optimization Weaknesses

Performance and timing code weaknesses in smart contracts usually affect the gas amount in the contracts. In the following, we define 2 code weaknesses belonging to this category.

Unbounded loops — TO#1 In Solidity, iterating through a potentially unbounded array of items can be costly, as exemplified in *getNotes()* in Listing 13. Since the array *notes* is provided as an input, the smart contract has no control over the maximum length, allowing a malicious actor to send in large arrays.

Creating subcontracts cost — TO#2 Contract deployments are very expensive operations. For instance, deploying a contract for every patient in Listing 13 is very costly. A malicious developer can use this weakness to cost the owner of the contract more Ether.

```
1 contract MedicalRecord {
2   struct Doctor {
3     bytes32 name;
4     uint id;}
5   struct Note {
6     bytes32 title;
7     bytes32 note;}
8   function getNotes()
9     view
10    public
11    isCurrentDoctor
12    returns (bytes32[], bytes32[])
13 {
14   bytes32[] memory titles = new bytes32[](notes.length);
15   bytes32[] memory noteTexts = new bytes32[](notes.length)
16   ;
17   for (uint i = 0; i < notes.length; i++) {
18     Note storage snote = notes[i];
19     titles[i] = snote.title;
20     noteTexts[i] = snote.note;
21   }
22   return (titles, noteTexts);}
```

²³ <https://blog.openzeppelin.com/on-the-parity-wallet-multisig-hack-405a8c12e8f7/>

Listing 13 Unbounded loops and creating subcontracts

Costly state variable data type — TO#3 Because of the padding rules, the `byte[]` data type consumes more gas than a byte array. In addition, declaring variables without constant consumes more gas than one declared with a constant. This weakness is also reported in Chen et al. (2020).

Costly function type — TO#4 A function declared public rather than external and not utilized within the contract consumes more gas on deployment than it should. This weakness is also reported in Chen et al. (2020).

5.1.8 Compatibility Weaknesses

This category is related to code weaknesses that prevent Ethereum from running normally on the developer machine. We find that the main root causes of this category are: (1) developers are not using the latest binaries/releases and (2) the hardware that is in use does not meet the minimum requirements. As this category played only a minor role in the analyzed posts, we did not label the posts in detail, but decided to keep this for future work.

5.1.9 Deployment and Configurations

This category of weaknesses is caused by wrong configurations and deployment.

Improper configuration — DL#1 Wrong or improper configuration of the smart contract application tool-chain can result in weaknesses or errors, which applies even if the contract itself is free of code weaknesses.

Violating contract size limit — DL#2 In Ethereum, limits are imposed by the gas consumed for the transaction. While there is no exact size limit there is a block gas limit and the amount of gas provided has to be within that limit. When deploying a contract, there are three factors to consider: (1) an intrinsic gas cost, (2) the cost of the constructor execution, and (3) the cost of storing the bytecode. The intrinsic gas cost is static, but the other two are not. The more gas consumed in the constructor, the less is available for storage. Normally, the vast majority of gas consumed is based on the size of the contract. If the contract size is large, the consumed gas can get close to the block gas size limits, preventing the deployment of the contract.

5.1.10 Authorization and Authentication

The following weaknesses directly affect the security of a smart contract and could enable attacks/exploits.

Lack of access control management — SV#1 Access control is an essential element to the security of a smart contract. Based on the privileges of each client/contractor party, there have to be strict rules implemented in the contract that enforce access control.

For example, the contract in Listing 14 is trying to provide functionality to whitelist addresses. The original function in line 10 does not have any access restrictions, meaning any caller can whitelist addresses. The modified version in line 15 only allows the contract owner to do so.

```

1 contract WHITELIST {
2     address owner; //set during the first call
3     modifier isOwner() {
4         require(msg.sender == owner);
5         _;
6     }
7     // insecure
8     function enableWhitelist(address address) {
9         //Whitelist an address
10    }
11    // secure
12    function enableWhitelist(address address) external
13    isOwner {
14        //Whitelist an address
15    }
16 }

```

Listing 14 Lack of access control management

Authorization via tx.origin — SV#2 tx.origin is a global variable in Solidity which returns the address of the account that sent the transaction. Rather than returning the immediate caller, tx.origin returns the address of the original sender (i.e., the first sender who initiated the transaction chain). It can make the contract vulnerable, if an authorized account calls into a malicious contract. Therefore, a call could be made to the vulnerable contract that passes the authorization check as tx.origin returns the original sender of the transaction, which in this case is the authorized account.

Signature based code weaknesses — SV#3 These code weaknesses are introduced as a result of insufficient signature information or weaknesses in signature generation and verification. Those include but not limited to:

- Lack of proper signature verification: One example can be relying on *msg.sender* for authentication and assuming that if a message is signed by the sender address, then it has also been generated by the sender address. Particularly in scenarios where proxies can be used to relay transactions, this can lead to code weaknesses. For more information, we refer to SWC-122²⁴
- Missing Protection against Signature Replay Attacks: To protect against Signature Replay Attacks, a secure implementation should keep track of all processed message hashes and only allows new message hashes to be processed. Without such control, it would be possible for a malicious actor to attack a contract and receive message hashes that were sent by another user multiple times. For more information, we refer to SWC-121²⁵
- Signature Malleability: The implementation of a cryptographic signature system in Ethereum contracts often assumes that the signature is unique, but signatures can be altered without the possession of the private key and still be valid. Valid signatures could be created by a malicious user to replay previously signed messages. For more information, we refer to SWC-117²⁶

5.1.11 Dependency and Upgradability Weaknesses

Dependencies of and upgrades to smart contracts can lead to a number of issues in smart contracts. We describe one of these weaknesses below.

²⁴ <https://swcregistry.io/docs/SWC-122>

²⁵ <https://swcregistry.io/docs/SWC-121>

²⁶ <https://swcregistry.io/docs/SWC-117>

Insecure contract upgrading — UV#1 There are two ways to upgrade a contract. First, to use a registry contract that keeps track of the updated contracts, and second, to split the contract into a logic contract and a proxy contract so that the logic contract is upgradable while the proxy contract is the same. Both approaches allow untrusted developers to introduce dependency code weaknesses in the updated contract's logic, allowing attackers to modify the logic of the upgraded contract using its dependencies, e.g. another contract. This code weakness was also reported in Atzei et al. (2017) and Chen et al. (2020).

5.1.12 Interface Weaknesses

This category describes weaknesses resulting in incorrect display of results. However, in these cases the smart contracts actually worked and produced the expected outcomes. The weaknesses were instead found in other applications that displayed the results. As these applications are not part of the smart contract, and there was no apparent code weakness in the contracts, we did not further investigate these kind of weaknesses.

5.2 How do frequency distributions compare across data sources? (RQ2)

To answer RQ2, we analyzed the frequency distributions of the defined eleven code weakness categories across the four data sources (i.e., Stack Overflow GitHub, CVE, and SWC). The frequency distributions of the defined categories are shown in Figs. 6 and 7 for the four data sources. Note that Fig. 7 (a) uses a different scale than the other three figures.

The resulting frequency distribution shows that the Language specific coding category and the Structural data flow category are the most common code weakness categories in Ethereum smart contracts

The language specific coding category is dominant on both Stack Overflow and GitHub. In contrast, structural data flow weaknesses are most frequent on CVE and SWC. Additionally,

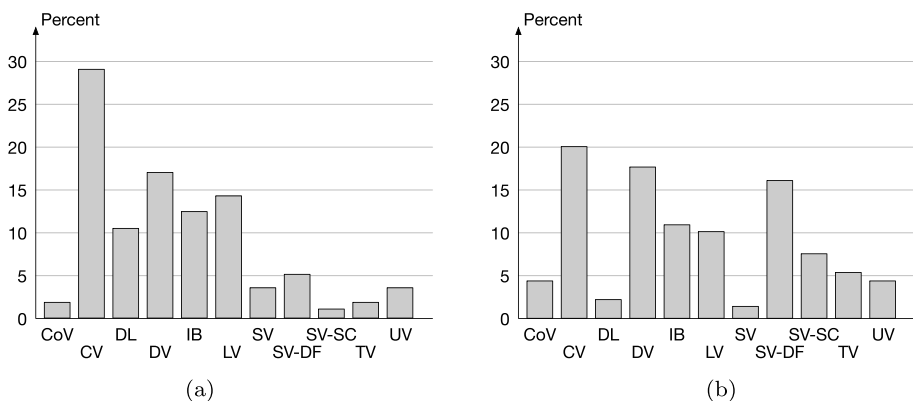


Fig. 6 Code Weakness Frequency Distribution in (a) StackOverflow and (b) GitHub

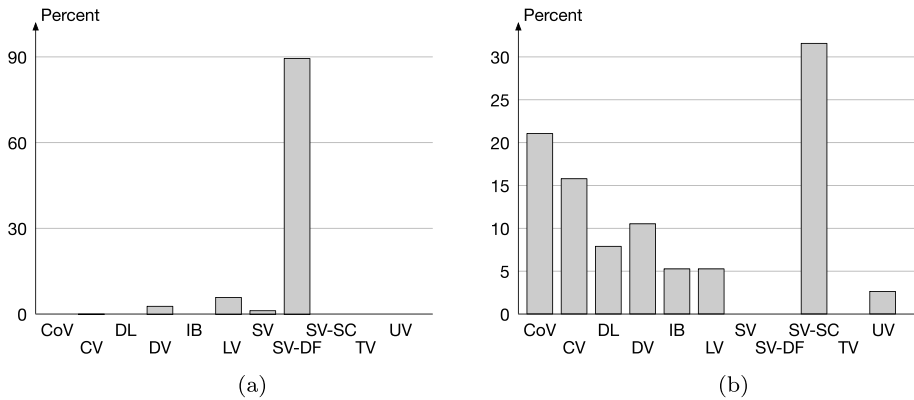


Fig. 7 Code Weakness Frequency Distribution in (a) CVE and (b) SWC

almost 80% of the reported structural data flow weaknesses in the NVD database (CVEs) are integer overflow/ underflow weaknesses. Interestingly, issues on Stack Overflow and GitHub appear to have similar frequency distributions. However, there is a statistically significant association between the frequency distributions and the data sources according to Fisher's exact test with an alpha value of 0.05 ($p=0.011$).

We further analyzed the variations of code weakness frequencies among the studied data sources. According to our analysis, by far the most frequent smart contract weaknesses in the CVE records fall under the Structural Data Flow (SV-DF) category and specifically under the arithmetic overflow/underflow category. This type of code weakness is common in software engineering. According to a report published on the CVE website²⁷, this code weakness has been among the top 10 weaknesses reported to CVE records for many years. Interestingly, SV-DF weaknesses are uncommon in Stack Overflow and on SWC.

5.3 What impact do the different categories of smart contract code weaknesses have? (RQ3)

In this section, we present the main impacts of vulnerabilities in smart contracts, thus answering RQ3. We unify all the proposed impact categories in the literature (i.e., Chen et al. (2020); Zhang et al. (2020)), and present a thorough classification scheme of code weakness impacts on Ethereum smart contracts. As discussed in Section 4, we followed a similar approach as in answering RQ1. The definitions of the final impact categories are depicted in Table 6, where the first seven categories focusing on the impact on the software product (I-D1) and the remaining categories are associated with the impact on business factors and contract owners (I-D2). Furthermore, Table 7 shows how the impact categories in literature related to our classification. Note that IP5 (in the second-last row) does not relate to any of our categories, as the category describes smart contracts that function as intended, something that we did not include in our classification.

²⁷ <https://cve.mitre.org/docs/vuln-trends/index.html>

Table 6 Classification Scheme of Impacts

Impact	Description	Short
Unexpected behavior	Contract behaves abnormally, e.g., generating incorrect output.	UB
Unwanted functionality	Contract executes wrong functionality because of wrong logic.	UF
Long response time	Long runtime of a smart contract to any input without providing the desired output.	LRT
Data Corruption	Data becomes unreadable, unusable or inaccessible, and unexpected output is generated.	DC
Memory disclosure	Problems in the memory storage of the smart contract.	MF
Poor performance	Non-optimal resource usage in terms of gas and time.	PPC
Unexpected stop	Unexpected exit and execution stop at the point of triggering the code weakness in the code.	USP
Information disclosure	When a code weakness is exploited, sensitive information is exposed to an actor not explicitly authorized to see it.	ID
Lost Ether or assets	An exploited code weakness can lead to unauthorized actors taking over the Ether of the contract and losing it.	LEA
Locked Ether	In the situation of triggering code weaknesses in a contract, one can lose access to the contract and lock the Ether in it without having access to it again.	LE
Lost control over the contract	By exploiting code weaknesses, an unauthorized actor can take over the contract.	LC

Our mapping shows that the impact of smart contract code weaknesses on certain aspects has not been examined in detail. For instance, the impact on the development process and the productivity of a software development team. Additional research in this area can quantify the extent to which the weaknesses impact smart contract development, the developing team, and the development of decentralized applications based on smart contracts.

Figure 8 presents the frequency distribution of code weakness impacts in smart contracts. In the process of analysing the impacts of various weaknesses, certain cases were encountered where determining the precise impact was challenging. As a result, these cases were designated as 'NF' (i.e., no impacts) due to the lack of clear and discernible impacts. The analysis of the impacts within the language specific coding category, as shown in Fig. 8(a), reveals that 'unexpected stop' is the most prevalent impact category. Similarly, in the data initialization and validation category as shown in Fig. 8(b), 'data corruption' emerges as the most prevalent impact category, followed closely by 'unexpected stop' as the second most prevalent impact. Moreover, the analysis of the impacts within the structural sequence category reveals a comparable impact distribution, specifically in the two impact categories of 'unexpected stop' and 'data corruption'. These three code weakness categories demonstrate

Table 7 Mapping literature-based Impact classifications to I-D

Category	UB	UF	LRT	DC	MF	PPC	USP	ID	LEA	LE	LC
Functionality* Zhang et al. (2020)		=									
Performance* Zhang et al. (2020)						=					
Security Zhang et al. (2020)	C*	C*	C*	C*	C*	C*	C*	C*	C*	C*	C*
Serviceability *Chen et al. (2020)	C*	C*	C*	C*	C*	C*	C*				
IP1 unwanted behaviors* Chen et al. (2020)	=										
IP3 non-exploitable UB* Chen et al. (2020)	*C										
IP4 Errors outside program call Chen et al. (2020)		*C									
IP5 No errors* Chen et al. (2020)											
IP2 UB without losses* Chen et al. (2020)	*C									*C	

C* indicates the corresponding category in our own classification is a subset of the corresponding category in the literature marked by *. *C means the category in the literature is a subset of our proposed category

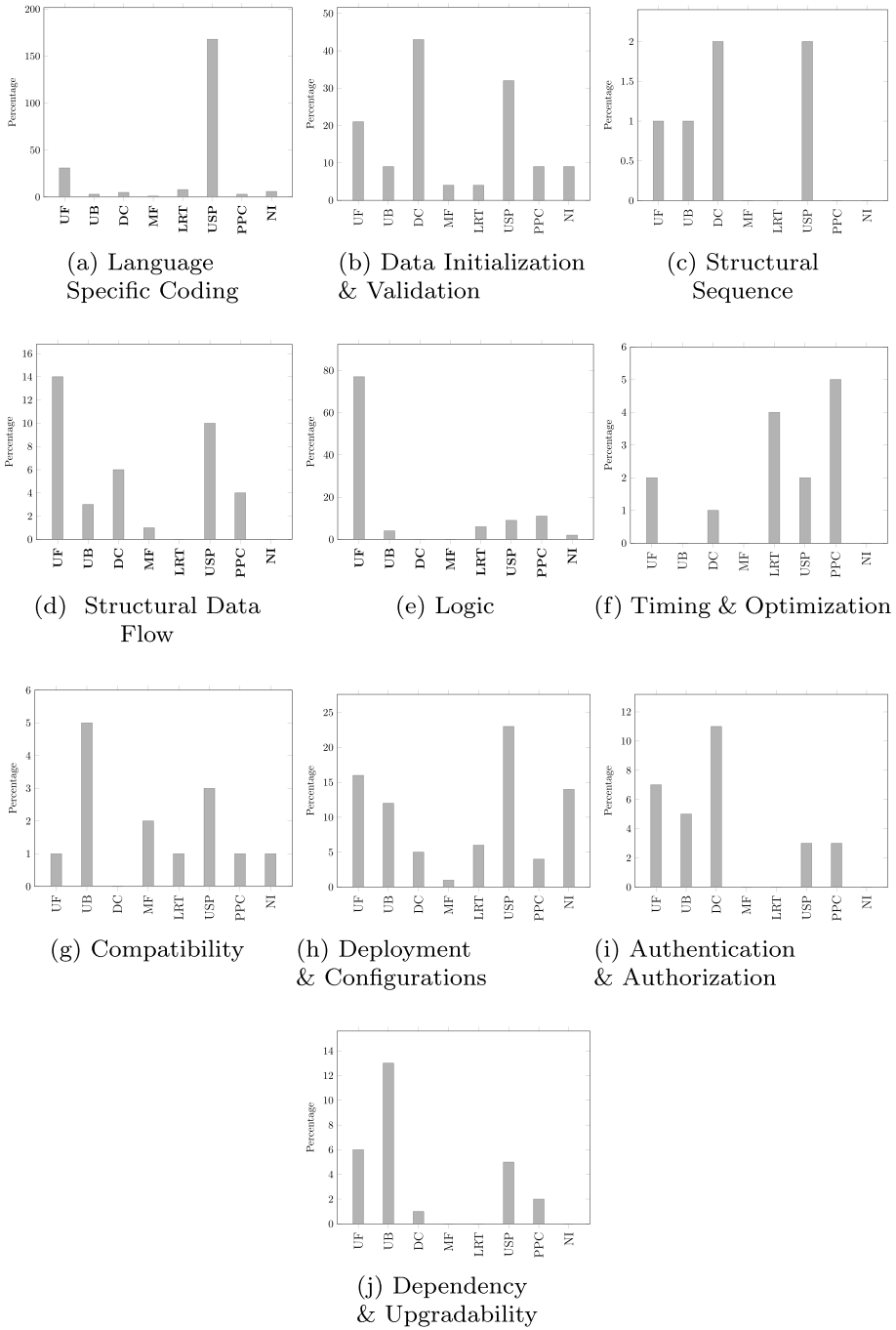


Fig. 8 Frequency Distribution of Code Weakness Impacts per Code Weakness Category in Smart Contracts

similar prevalence, emphasizing the importance of promptly addressing the impacts associated with these weaknesses and implementing effective remediation measures. In Fig. 8 (d) and (e), the analysis of the structural data flow and logic categories reveals that 'unwanted functionality' is the most prevalent impact within these respective categories. This finding emphasizes the importance of addressing and mitigating the risks associated with unwanted functionality to ensure the robustness of smart contracts. The Timing and Optimization category encompasses two significant impacts: poor performance and long response time as shown in Fig. 8 (f). These impacts are primarily attributed to the extensive utilization of resources, such as gas or time. The deployment & configurations category also demonstrates the prevalent impact of long response time as shown in Fig. 8 (h). Additionally, the compatibility and dependency & upgradability categories highlights the dominant impact category of unexpected behavior. Furthermore, within the authentication & authorization category, data corruption emerges as the most prevalent impact category, followed closely by unwanted functionality as the second most frequent category. These findings emphasize the importance of robust authentication and authorization mechanisms to mitigate the legal and technical risks associated with these impacts. Finally, we did not analyse the impacts of the Interface category, given that the identified weaknesses were attributed to external applications and not directly related to the smart contracts themselves as mentioned before.

6 Discussion

In the following, we discuss our findings in terms of implications and relation to existing work.

We find a substantial number of code weaknesses being discussed in social coding platforms and existing vulnerability repositories. This clearly shows that this is an important topic to study and analyze. Because of the unique characteristics of smart contracts, e.g., immutability and gas consumption, it is important to make sure that code weaknesses with severe impacts are fixed or even detected before deploying the contract to the blockchain.

As demonstrated in our findings, existing classifications either focus on a single dimension of smart contract code weaknesses, such as the error source, or mix multiple dimensions in a single classification. Our mapping unifies these different dimensions to some extent and shows how different classification schemes relate.

In addition to mixing dimensions, the majority of existing classification schemes for smart contract code weaknesses include broad categories, such as security and availability, to which many code weaknesses can be assigned. As such, these categories do not support reasoning about the included code weaknesses, which is an important quality criterion for classifications (Ralph 2018). Furthermore, broad categories might prevent orthogonality of the categories, i.e., that a single code weakness fits into a single category only. In the example mentioned above, i.e., security and availability, many code weaknesses can lead to negative effects on both, and thus could be labeled both.

The frequency distributions discussed in Section 5.2 show notable differences between the frequencies of found code weaknesses across different data sources. This highlights that focusing on a single source biases the resulting study. It further demonstrates that established databases such as CVE and SWC do not reflect well the topics discussed in public coding platforms such as Stack Overflow or GitHub. On CVE and SWE, it appears that reported weaknesses are primarily those that are well-known and were discovered as results of known

attacks. For instance, the re-entrancy attack was discovered when the infamous DAO attack took place in 2016 Mehar et al. (2019). On Stack Overflow and GitHub, we observe specifically that developers seem to have a poor understanding of pre-defined functions such as *view* and *pure*, and that it is hard for them to cope with continuous changes and updates in Solidity and its documentation.

On the one hand, this finding suggests that tools for verification and analysis of smart contracts is of high importance, especially focusing on the prevalent code weakness categories in our classification. On the other hand, the observed frequencies might only be a symptom of the technology maturity. Hence, these issues might become less prevalent once Solidity matures and updates become less frequent. Finally, it is important to note that SWC and CVE both are aggregating weaknesses based on reports, i.e., are derived data sources. This makes direct comparison of frequencies to those on Stack Overflow and GitHub challenging, and might explain the differences at least in part.

Moreover, we studied the impact of smart contract code weaknesses, as these impacts can uncover significant risks in smart contracts and blockchain-based applications. Understanding the proposed dimensions and the impacts of smart contract code weaknesses per dimension is also crucial for developing effective strategies to prioritize, prevent, detect, and mitigate such code weaknesses in smart contracts. We defined 11 impacts that are divided to two main dimensions. When examining the first impact dimension (i.e., I-D1), several key impacts stand out, including unexpected stoppage, memory disclosure, data corruption, long response times, unwanted functionality, and unexpected behavior. While some of these impacts are common across different software systems, there are notable differences in the context of smart contracts. Unexpected stoppage, for instance, can be disruptive in any software application, causing crashes or halting normal operations. Similarly, data disclosure and memory corruption can compromise the integrity of sensitive information. These impacts are shared by both smart contracts and other software.

However, there are distinct aspects to consider in the context of smart contracts. Smart contracts operate on decentralized platforms such as blockchains, which introduce unique challenges. The long response times of a smart contract can disrupt the execution of transactions within a decentralized network and hinder the functioning of decentralized applications (DApps) relying on them. This distributed nature and the reliance on consensus mechanisms in smart contracts differentiate their long response times from that of other software systems. Moreover, it can affect user experience and system performance in both smart contracts and other software applications. Nevertheless, in the case of smart contracts, it may be more pronounced since they often involve financial transactions or contractual legal agreements. Delays in processing transactions due to long response times can impact the timeliness of these transactions, potentially resulting in financial losses or legal disputes.

In addition, code weaknesses in smart contracts can have significant business-related impacts and directly affect contract owners (i.e., I-D2). These impacts include information disclosure, lost control over the contract, lost Ether or assets, and locked Ether. For example, the loss of Ether or assets is a critical business impact resulting from code weaknesses in smart contracts, particularly when financial transactions or digital asset management are involved. Recovering lost assets can be challenging due to the immutability of smart contracts, potentially requiring costly dispute resolution mechanisms or legal interventions. Another impact is locked Ether, which occurs when Ether becomes inaccessible or nontransferable due to code weaknesses in the contract. This situation can arise when a smart contract is designed to hold or manage funds, but experiences a bug that prevents the contract owner

from accessing their assets. Locked Ether can have severe financial implications, hindering the contract owner's ability to manage or utilize their funds as intended. Lastly, the loss of control over the contract is another consequential impact. When a code weakness compromises the logic or execution of a smart contract, the contract owner may lose control over its operations. This loss of control can have cascading effects, affecting contractual agreements, business relationships, and even financial viability. Regaining control and rectifying the code weakness may require technical expertise, collaboration with stakeholders, and potentially renegotiating affected agreements.

Based on our findings, we can provide a number of recommendations to researchers and practitioners to improve the development of smart contracts.

First, available smart contract static and dynamic detection tools must urgently target the defined categories of code weaknesses in this paper, specifically categories that are under-represented in SWC and CVE. Our breakdown of the frequencies at which the different categories occur can help prioritize this development to target the most important categories first. Second, software engineers and Solidity language designers should work together to define patch templates for known categories of code weaknesses in smart contracts. Patch templates can be applied based on the category of code weaknesses that are discovered. These templates can be implemented using each of the defined categories' root causes in this paper. Third, there is a need to define coding best practices for smart contracts and make them available to developers to avoid some language specific coding weaknesses. Software engineers are urged to follow the coding standards and recommendations by Solidity language designers, which can improve contract quality and reduce unexpected gas loss. For instance, some implementation changes on Solidity token standards (i.e., ERC-20) can result in code weaknesses.

Based on the defined categories, software engineers can implement methods to detect weaknesses in smart contracts. For example, to detect a logic code weakness category in a smart contract, software engineers can implement a sandbox to study the behavior of the contract before deployment. Also, to detect language specific coding code weaknesses, implementing a gas calculator and adding it to static analyzers can help owners estimate the amount of gas the contract will cost them.

Based on the analyzed data from the four data sources, we believe smart contract coding recommendations should include, e.g., avoiding multidimensional arrays, or arrays in general; carefully checking gas consumption amounts; carefully checking the gas cost for each predefined function; avoid deploying big sized contracts; using specific safe libraries such as SafeMath²⁸ to avoid common pitfalls such as underflows; and avoiding substantial subcontract creation. We also see a need for action from the Solidity team, in particular when it comes to clearly defining gas requirements for language specific coding category including language predefined functions, declaration types, creating subcontracts, upgrading contracts, and other language artifacts and operations. Such documentation could substantially contribute to a reduction in smart contract weaknesses in Solidity/Ethereum.

Considering the impacts of smart contract code weaknesses discussed earlier, there is a clear need for the development of automated solutions that can prioritize the resolution of code weaknesses based on their potential impacts. These solutions can assist developers in addressing significant vulnerabilities, mitigating the risk of financial losses. By focusing on resolving critical weaknesses first, developers can ensure that their smart contracts are more

²⁸ <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/utils/math/SafeMath.sol>

robust and resilient, thereby protecting the interests of contract owners and users. In addition, we encourage researchers to explore the implementation of monitoring and updating mechanisms for deployed smart contracts to proactively address emerging threats and vulnerabilities, promptly responding to reported impacts. Furthermore, it is crucial for developers and practitioners to prioritize secure coding practices in smart contract development. By promoting awareness of the potential impacts of code weaknesses and providing education on secure coding techniques, developers can enhance the overall security posture of smart contracts. This includes implementing best practices such as input validation, access control, and secure data handling, which can significantly reduce the likelihood of code weaknesses and their associated impacts.

Moreover, it is important to implement maintenance measures to ensure the continued effectiveness of the proposed taxonomies. Several key measurements can be considered in this regard. First, regular updates are crucial to keep the taxonomy up to date with emerging code weaknesses in smart contracts and their impacts. For instance, AutoMESC, an automatic framework for mining and classifying Ethereum smart contract code weaknesses and their fixes (Soud et al. 2022), continuously collects newly published code weaknesses from open smart contract projects on GitHub and reported code weaknesses from CVE. This continuous data collection ensures that the code weaknesses taxonomy remains relevant and encompasses the latest smart contract code weaknesses. Additionally, periodic reviews and revisions by experts in the field further contribute to keeping the taxonomies up-to-date and accurate in capturing the ever-changing landscape of smart contract code weaknesses. Second, actively engaging smart contract developers, auditors, and researchers in the taxonomy's maintenance process allows for tapping into their diverse insights and experiences, leading to a more robust classification system. Finally, mapping historical incidents to the taxonomy's code weaknesses helps to continually assess its accuracy and coverage. We believe that implementing these measures will contribute to the ongoing relevance and usefulness of the taxonomy in addressing smart contract code weaknesses and their impacts.

Finally, as the proposed taxonomies evolve, several challenges may arise when scholars and experts attempt to enrich the taxonomy in real-world scenarios. For instance, enriching the taxonomy based on real-world smart contracts extracted from Ethereum, Stack Overflow posts, OSS projects on GitHub, and other sources presents unique challenges. One such challenge is that information about vulnerabilities or their impact in each source (i.e., GitHub, Stack Overflow) can vary significantly, making it difficult to enrich the vulnerability and impact taxonomies with the required information per vulnerability. Another significant challenge we expect is that the same vulnerability might have different names or labels within different communities, such as GitHub or Stack Overflow. This can make it hard to categorize vulnerabilities consistently. One strategy to overcome this challenge is distinguishing the root cause, which is imperative to ensure the effectiveness and accuracy of the taxonomy. Additionally, addressing the impact of vulnerabilities in smart contracts poses a significant challenge. Comprehending the impacts of each vulnerability is complicated due to the complexities of smart contract behavior on the blockchain and the potential for cascading effects. Several strategies can be employed to tackle this challenge, such as considering only the impact listed in the vulnerability description, replicating the vulnerability to trace its effects, and seeking expert opinions for accurate impact classification. Lastly, the introduction of a new category into the taxonomy presents a challenge. When a new category is added, it becomes essential to review all existing vulnerabilities and determine if any should be placed within this new category. This review process ensures that vulnerabilities are accurately categorized.

7 Evaluation

In this section, we evaluate our proposed classification scheme using recommendations from Usman et al. (2017). At this point, we evaluate by line of argumentation. Additionally, we are currently performing an empirical evaluation with smart contract experts, which is beyond the scope of this paper.

Initially, we discuss the utility of the classification scheme, i.e., to what extent it is fit for the intended purpose. After that, we discuss benchmarking, i.e., compare our classification scheme with existing ones in the same area. Then, we argue that our classification scheme is orthogonal in its different facets, i.e., that classes of the classification do not overlap.

Finally, we outline the methodology that we intend to employ for our empirical evaluation.

7.1 Utility

The main purpose of the proposed classification scheme is to enable a detailed analysis of smart contract weaknesses based on their root causes and impact. We constructed this classification based on weakness databases (SWC and CVE) and real-life smart contracts or excerpts thereof. As such, utility is given by construction, since we show that actual smart contract weaknesses could be classified in the proposed way. By using a substantial sample of weaknesses from GitHub and Stack Overflow, we are further confident that the classification is general enough that it applies to smart contract weaknesses written in Solidity, beyond the sample used for creating it.

7.2 Benchmarking

Our classification is developed by using data from real-life contracts and vulnerabilities databases. Furthermore, we map our categories to existing classifications. While it cannot be expected that our classification is better in every aspect, we argue that it does not suffer from shortcomings observed in the existing classifications, in particular the use of overly broad classes. In addition, our classification is more detailed than existing ones, thus allowing for more detailed analyses of smart contract weaknesses.

7.3 Orthogonality

In our classification, categories are orthogonal, as they have definitions and main root causes that do not overlap. For example, logic code weaknesses can be inspected by following the target logic for which the contract was constructed, whereas language-specific coding categories should be inspected by following the rules and recommendations of the target language; i.e., they cannot be inspected in the same way as logic code weaknesses. Similarly, each of the other categories has its own specific characteristics and elements. For instance, there are specific methods to check interface code weaknesses and these are different from the ones that check authorization and authentication in the contract. Accordingly, we conclude that the classification is orthogonal by design.

For our empirical evaluation, we will systematically assess five main aspects of our proposed smart contract vulnerability and impact taxonomies: taxonomy importance, relevance, comprehensiveness, effectiveness, and practicality. Our evaluation methodology comprises several key phases. Initially, we will collect data by conducting focus groups and interviewing

experts from leading companies in smart contracts and blockchain. These focus groups will serve as platforms for in-depth discussions, where experts can share their perspectives on the aforementioned aspects of our proposed taxonomies. The multidisciplinary nature of these focus groups will ensure a holistic evaluation, considering viewpoints from academia and industry. During these sessions, experts will be presented with the taxonomies, dimensions, and classified vulnerability types, encouraging them to critically analyze potential strengths and limitations. Following data collection, we will conduct a comprehensive analysis, which involves categorizing and structuring the data. Subsequently, we will draw meaningful conclusions from the data and engage in a thorough discussion of our findings. This process entails identifying patterns and correlations within the data, enabling us to assess the effectiveness of our taxonomies in categorizing smart contract vulnerabilities and their impacts.

8 Threats to Validity

In the following, we discuss threats to the validity of our findings according to internal and external validity, as well as reliability.

8.1 Internal Validity

The open card sorting was used as the main method to categorize and label the posts in this work. This method is subjective and open to bias, hence two experts verified the manual labeling twice independently in order to mitigate this threat. In case of any disagreement, the two experts would discuss it until reaching to a consensus. However, we acknowledge that qualitative inquiry cannot be entirely objective.

The labeling process was time intensive. We did not account for fatigue factors, but relied on the experts doing the work in shorter iterations. Hence, there is a potential threat to validity that fatigue might have affected the quality of the labels.

An additional threat to internal validity lies in the keyword search employed during data collection. For instance, to collect Q&A posts from Stack-Overflow, we used the tags “Smart contract”, “Ethereum”, and “Solidity”, while for GitHub we used “smart contract”, “Solidity”, and “Ethereum”. Also, a part of the collected data with insufficient information or with information that is not related to smart contract code weaknesses written in Solidity were excluded from the analysis. This search and filtering process might have led to exclude relevant data that could have led to further insights. To mitigate this threat, we collected the data using the keywords that resulted in the maximum number of records. We also made the data publicly available so researchers and developers can verify them.

As trustworthiness of the collected posts can be a source of noise in the data, we removed negative voted questions from the dataset. This might have resulted in a systematic under-representation of certain types of code weaknesses. Given the maturity of Solidity, we deemed this step warranted to allow for data of sufficiently high quality.

8.2 External Validity

A threat to external validity is the continuous updates of Ethereum using the hard fork. More improvements are being added to Ethereum with each hard fork such as the EIPs ²⁹

²⁹ <https://github.com/ethereum/EIP>

that ensures the energy efficiency of the proof-of-work. Furthermore, many new features are added to Solidity newer versions. Therefore, new contract code weaknesses may be introduced and some others may be resolved. This means that our classification might not generalize to newer (or older) versions of Solidity.

Focusing on Solidity and the Ethereum blockchain only limits the external validity of the results, as there are other blockchains and other smart contract languages that could have yielded further or different code weaknesses. We focus on Ethereum since it is the second-largest blockchain, and the largest blockchain that supports smart contracts (written in Solidity). However, additional studies into other technologies could be a valuable addition to our findings, and our results might not generalize to other blockchains or smart contract languages.

Finally, some defined weaknesses can be considered as controversial. As an example, greedy contracts are those that only receive Ether and do not send it. While this can be considered a weakness in many cases, there might be cases where the contract owner intended it, e.g., for an online game that only receives Ether as in-game purchase. Similarly, hard-coded address is a well-known code weakness. To minimize this threat, we clarified in the definition of each weakness when the weakness applies and indeed results in unexpected behavior. A further issue might arise if weaknesses are coupled with other weaknesses, resulting in unpredictable behavior. This remains a future direction and an opportunity to study the behavior of these weaknesses in combination with others.

8.3 Reliability

To ensure reliability of the results, we described in detail the data collection and analysis process. We systematically calculated inter-rater reliability coefficients in iterative rounds, to ensure sufficiently clear categories and agreement among two expert raters. Finally, we published the full dataset (Soud et al. 2021). Overall, these steps should ensure reliability and enable replication of our study.

9 Conclusion

Due to the immaturity of blockchain technology, code weaknesses in smart contracts can have severe consequences and result in substantial financial losses. Hence, it is essential to understand code weaknesses of smart contracts. However, there exist several shortcomings in existing classifications, i.e., they focus on single dimensions, mix dimensions, propose too broad categories, or rely on single data source omitting important sources for code weaknesses. To address this gap, we extracted smart contract code weaknesses written in Solidity from a number of important data sources, classified them in terms of error source and impact, and related them to existing classifications in literature. Our findings show that language-specific coding and structural data flow are the dominant categories of code weaknesses in Solidity smart contracts. We also find that many code weaknesses are similar to known issues in general purpose programming languages, such as integer overflow and erroneous memory management. However, the immaturity and rapid evolution of the technology and the Solidity language, and the added concept of gas furthermore adds code weaknesses, and increases the risk of attacks and financial losses. Interestingly, we find that the frequency at which the different categories occur differs widely across data sources, indicating that they should not be viewed in isolation. On a meta level, these frequency distributions can present a valuable

source of information to reason about biases in studies that only use one or a few of these sources.

Our classification scheme is a further step to standardize and unify code weakness analysis in smart contracts. This can support researchers in building tools and methods to avoid, detect, and fix smart contract code weaknesses in the future.

Specifically, we see a number of directions for future research. First, we are currently in the process of performing an empirical evaluation of the taxonomy with experts in smart contracts and in security, in particular of the utility of the taxonomy for practical use. Second, future studies should investigate whether our classification scheme can be generalized by investigating other smart contracts in other blockchain networks, e.g., Hyperledger, Stellar, and Openchain. Potentially, our classification needs to be modified to fit into other networks or languages. Similarly, the classification might have to be adapted as Solidity evolves and as new languages are developed for Ethereum smart contracts. Third, more dimensions and characteristics of the studied code weaknesses can be explored in the future. For instance, patterns among the extracted code weaknesses could be abstracted, as well as the evolution over time. For each of the categories, code metrics and detection tools can be explored. Based on our results, future work should investigate in more depth why frequency distributions differ across data sources. Additionally, a valuable contribution would be to study how the weakness categories are connected with the impact categories, i.e., which weaknesses typically have which impact(s). It should be investigated in which ways the defined code weaknesses can be exploited, as well as what the impact of these exploits will be. This can be accomplished by developing automated scripts or manually devising such exploits. Finally, Ethereum merge has happened very recently with multiple significant changes, e.g., the protocol has been updated from “proof-of-work” to “proof-of-stake”. This might affect existing code weaknesses and the proposed taxonomy. Thus, a follow-up study should be conducted to assess the impact of this merge on the classification, also in order to assess the robustness of our classification for future merges.

Acknowledgements The authors would like to thank Mohammad Alsarhan, a security expert, for participating in the card sorting and inter-rater agreement discussions. This work was supported by the Icelandic Research Fund (Rannís) grant number 207156-051.

Data Availability The datasets generated during and/or analysed during the current study are available in the Zenodo repository, <https://doi.org/10.5281/zenodo.6388179>.

Declarations

Competing Interests The authors declared that they have no conflict of interest.

References

- Ahasanuzzaman M, Asaduzzaman M, Roy CK, Schneider KA (2016) Mining duplicate questions of stack overflow. In: 2016 IEEE/ACM 13th Working conference on mining software repositories (MSR), pp. 402–412. IEEE
- Alharby M, Aldweesh A, van Moorsel A (2018) Blockchain-based smart contracts: A systematic mapping study of academic research (2018). In: 2018 International conference on cloud computing, big data and blockchain (ICCB), pp 1–6. IEEE
- Alharby M, Van Moorsel A (2017) Blockchain-based smart contracts: A systematic mapping study. arXiv preprint [arXiv:1710.06372](https://arxiv.org/abs/1710.06372)

- Atzei N, Bartoletti M, Cimoli T (2016) A survey of attacks on ethereum smart contracts. *IACR Cryptol. ePrint Arch.* 2016:1007
- Atzei N, Bartoletti M, Cimoli T (2017) A survey of attacks on ethereum smart contracts (sok). In: *International conference on principles of security and trust*, pp 164–186. Springer
- Ayman A, Aziz A, Alipour A, Laszka A (2019) Smart contract development in practice: Trends, issues, and discussions on stack overflow. *arXiv preprint arXiv:1905.08833*
- Ayman A, Roy S, Alipour A, Laszka A (2020) Smart contract development from the perspective of developers: Topics and issues discussed on social media. In: *Cryptography Financial, Security Data* (eds) Bernhard M, Bracciali A, Camp LJ, Matsuo S, Maurushat A, Rønne PB, Sala M. Springer International Publishing, Cham, pp 405–422
- Badawi E, Jourdan GV (2020) Cryptocurrencies emerging threats and defensive mechanisms: A systematic literature review. *IEEE Access*
- Bajaj K, Pattabiraman K, Mesbah A (2014) Mining questions asked by web developers. In: *Proceedings of the 11th working conference on mining software repositories*, pp 112–121
- Bhat M, Vijayal S (2017) A probabilistic analysis on crypto-currencies based on blockchain. In: *2017 International conference on next generation computing and information systems (ICNGCIS)*, pp 69–74. IEEE
- Calefato F, Lanubile F, Marasciulo MC, Novielli N (2015) Mining successful answers in stack overflow. In: *2015 IEEE/ACM 12th Working conference on mining software repositories*, pp 430–433. IEEE
- Chen C, Xing Z (2016) Mining technology landscape from stack overflow. In: *Proceedings of the 10th ACM/IEEE international symposium on empirical software engineering and measurement*, pp 1–10
- Chen H, Pendleton M, Njilla L, Xu S (2020) A survey on ethereum systems security: Vulnerabilities, attacks, and defenses. *ACM Comput Surv (CSUR)* 53(3):1–43
- Chen J, Xia X, Lo D, Grundy J, Luo X, Chen T (2020) Defining smart contract defects on ethereum. *IEEE Trans Softw Eng*
- Cosentino V, Izquierdo JLC, Cabot J (2017) A systematic mapping study of software development with github. *IEEE Access* 5:7173–7192
- Daian P (2016) Analysis of the dao exploit. *Hacking, Distributed*, p 6
- Dingman W, Cohen A, Ferrara N, Lynch A, Jasinski P, Black PE, Deng L (2019) Defects and vulnerabilities in smart contracts, a classification using the nist bugs framework. *Int J Netw Distrib Comput* 7(3):121–132
- Durieux T, Ferreira JF, Abreu R, Cruz P (2020) Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In: *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, pp 530–541
- Ethereum: Solidity by example (2021). <https://docs.soliditylang.org/en/latest/solidity-by-example.html>
- Ethereum Improvement Proposals (EIPs): EIP-1470: Smart Contract Weakness Classification (SWC). (2020) <https://github.com/ethereum/EIPs/issues/1469>. [Online] Accessed 01 April 2020
- Hewa T, Yliantila M, Liyanage M (2020) Survey on blockchain based smart contracts: Applications, opportunities and challenges. *J Netw Comput Appl* 102857
- Huang Y, Bian Y, Li R, Zhao JL, Shi P (2019) Smart contract security: A software lifecycle perspective. *IEEE Access* 7:150184–150202
- Idelberger F, Governatori G, Riveret R, Sartor G (2016) Evaluation of logic-based smart contracts for blockchain systems. In: *International symposium on rules and rule markup languages for the semantic web*, pp 167–183. Springer
- Kalra S, Goel S, Dhawan M, Sharma S (2018) Zeus: Analyzing safety of smart contracts. In: *Ndss*, pp 1–12
- Khan SN, Loukil F, Ghedira-Guegan C, Benkhelifa E, Bani-Hani A (2021) Blockchain smart contracts: Applications, challenges, and future trends. *Peer-to-peer Netw App* 14(5):2901–2925
- Khan ZA, Namin AS (2020) Ethereum smart contracts: Vulnerabilities and their classifications. In: *2020 IEEE International conference on big data (big data)*, pp 1–10. IEEE
- Khan ZA, Namin AS (2020) A survey on vulnerabilities of ethereum smart contracts. *arXiv preprint arXiv:2012.14481*
- Maiden N (2009) Card sorts to acquire requirements. *IEEE Softw* 26(3):85–86
- Soud M, Liebel G, MH (2021) Dataset: A fly in the ointment: An empirical study on the characteristics of Ethereum smart contracts code weaknesses and vulnerabilities. <https://doi.org/10.5281/zenodo.4441254>. <https://doi.org/10.5281/zenodo.6388179>
- Mavridou A, Laszka A, Stachtari E, Dubey A (2019) Verisolid: Correct-by-design smart contracts for ethereum. In: *International conference on financial cryptography and data security*, pp 446–465. Springer
- Mehar MI, Shier CL, Giambattista A, Gong E, Fletcher G, Sanayhie R, Kim HM, Laskowski M (2019) Understanding a revolutionary and flawed grand experiment in blockchain: the dao attack. *J Case Inform Technol (JCIT)* 21(1):19–32
- Nawaz A (2012) A comparison of card-sorting analysis methods. *10th Asia Pacific conference on computer human interaction (Apchi 2012)*. Matsue-city, Shimane, Japan, pp 28–31

- Norvill R, Fiz B, State R, Cullen A (2019) Standardising smart contracts: Automatically inferring ERC standards. In: 2019 IEEE International conference on blockchain and cryptocurrency (ICBC), pp 192–195. IEEE
- Perez D, Livshits B (2019) Smart contract vulnerabilities: Vulnerable does not imply exploited. arXiv preprint [arXiv:1902.06710](https://arxiv.org/abs/1902.06710)
- Permenev A, Dimitrov D, Tsankov P, Drachsler-Cohen D, Vechev M (2020) Verx: Safety verification of smart contracts. In: 2020 IEEE symposium on security and privacy (SP), pp 1661–1677. IEEE
- Philosophy G (2019) Solidity by example. https://consensys.github.io/smart-contract-best-practices/general_philosophy/
- Ponzanelli L, Bavota G, Di Penta M, Oliveto R, Lanza M (2014) Mining stackoverflow to turn the IDE into a self-confident programming prompter. In: Proceedings of the 11th working conference on mining software repositories, pp 102–111
- Praitheeshan P, Pan L, Yu J, Liu J, Doss R (2019) Security analysis methods on ethereum smart contract vulnerabilities: a survey. arXiv preprint [arXiv:1908.08605](https://arxiv.org/abs/1908.08605)
- Ralph P (2018) Toward methodological guidelines for process theories and taxonomies in software engineering. IEEE Trans Softw Eng 45(7):712–735
- Rameder H (2021) Systematic review of ethereum smart contract security vulnerabilities, analysis methods and tools. Ph.D. thesis, Wien
- Rugg G, McGeorge P (1997) The sorting techniques: a tutorial paper on card sorts, picture sorts and item sorts. Expert Syst 14(2):80–93
- Sakai R, Aerts J (2015) Card sorting techniques for domain characterization in problem-driven visualization research. In: EuroVis (Short Papers), pp 121–125
- Samreen NF, Alalfi MH (2021) A survey of security vulnerabilities in ethereum smart contracts. arXiv preprint [arXiv:2105.06974](https://arxiv.org/abs/2105.06974)
- Sánchez DC (2018) Razel: Private and verifiable smart contracts on blockchains. arXiv preprint [arXiv:1807.09484](https://arxiv.org/abs/1807.09484)
- Seacord RC, Householder AD (2005) A structured approach to classifying security vulnerabilities. Carnegie-mellon university pittsburgh pa software engineering inst, Tech. rep
- Sillaber C, Waltl B, Treiblmaier H, Gallersdörfer U, Felderer M (2021) Laying the foundation for smart contract development: an integrated engineering process model. IseB 19(3):863–882
- Soud, M., Qasse, I., Liebel, G., Hamdaqa, M (2022) Automesc: Automatic framework for mining and classifying ethereum smart contract vulnerabilities and their fixes. arXiv preprint [arXiv:2212.10660](https://arxiv.org/abs/2212.10660)
- Spencer D (2009) Card sorting: Designing usable categories. Rosenfeld Media
- Usman M, Britto R, Börstler J, Mendes E (2017) Taxonomies in software engineering: A systematic mapping study and a revised taxonomy development method. Inf Softw Technol 85:43–59
- Vacca A, Di Sorbo A, Visaggio CA, Canfora G (2021) A systematic literature review of blockchain and smart contract development: Techniques, tools, and open challenges. J Syst Softw 174:110891
- Vegas S, Juristo N, Basili VR (2009) Maturing software engineering knowledge through classifications: A case study on unit testing techniques. IEEE Trans Softw Eng 35(4):551–565
- Viera AJ, Garrett JM et al (2005) Understanding interobserver agreement: the kappa statistic. Fam med 37(5):360–363
- Wan Z, Xia X, Lo D, Chen J, Luo X, Yang X (2021) Smart contract security: A practitioners' perspective. In: 2021 IEEE/ACM 43rd International conference on software engineering (ICSE), pp 1410–1422. IEEE
- Wood JR, Wood LE (2008) Card sorting: current practices and beyond. J Usability Stud 4(1):1–6
- Yamashita K, Nomura Y, Zhou E, Pi B, Jun S (2019) Potential risks of hyperledger fabric smart contracts. In: 2019 IEEE International workshop on blockchain oriented software engineering (IWBOSE), pp 1–10. IEEE
- Zhang P, Xiao F, Luo X (2020) A framework and dataset for bugs in ethereum smart contracts. In: 2020 IEEE International conference on software maintenance and evolution (ICSME), pp 139–150. IEEE
- Zheng Z, Xie S, Dai HN, Chen W, Chen X, Weng J, Imran M (2020) An overview on smart contracts: Challenges, advances and platforms. Futur Gener Comput Syst 105:475–491

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.